Algorithms and Processes of a No-Limit Hold'em Engine

Our first-principles reference on poker rules, probability, game theory, engine processes, associated evaluators and solvers, and underlying mathematics



We build and verify the mathematical foundations, state-machine rules, evaluation algorithms, probability estimators, and game-theoretic decision policies of our 4- to 8-player No-Limit Texas Hold'em poker engine from first principles.

Contents

1 Introduction to Poker, Probability, and Game Theory **1**
1.1 What Is a Poker Engine? Our Overview of the Core Processes . . . 1
1.2 Introduction to Probability from Scratch . . . 1
1.3 Introduction to Game Theory from Scratch . . . 2

2 Rules, Hand Rankings, Pot Size, and Core Terminologies **3**
2.1 The Deck, 4–8 Player Positions, Blinds, and Street Progression . . . 3
2.2 Legal Betting Actions and Raising Rules . . . 4
2.3 Complete Five-Card Hand Rankings and Combinatorial Frequencies . . . 5
2.4 Pot Size, Pot Odds, Stack-to-Pot Ratio, and Side Pots . . . 6
2.5 Comprehensive Terminology Reference . . . 6

3 Process I: Card Representation, Cryptographic Shuffling, and Enclave Sealing **6**
3.1 The Bijective Integer Encoding Process . . . 6
3.2 The Unbiased Cryptographic Shuffling Process . . . 7
3.3 The Closure-Sealed Dealer Enclave and Provably Fair Commitment Process (DEALERVAULT) . . . 8

4 Process II: Hand Evaluation and Associated Evaluator Architectures **9**
4.1 The Base-13 Lexicographical Packing Process . . . 9
4.2 The Five-Card Classification Process (EVAL5) . . . 10
4.3 The 21-Subset Seven-Card Evaluation Process (BEST7) . . . 11
4.4 The Single-Pass Bitmask and Lookup-Table Evaluation Process (RANK7FAST) . . . 11
4.5 Processes in Associated Standard Hand Evaluators . . . 13

5 Process III: Equity Estimation, Multi-Way Bunching, and Runout Enumeration **13**
5.1 Starting-Hand Isomorphism (1,326 to 169 Classes), Card Removal, and Outs . . . 13
5.2 The Monte Carlo Rollout Process (EQUITY) and Bessel-Corrected Standard Error . . . 15
5.3 Range Rejection Sampling, Multi-Way Bunching, and Board-Lock Protection . . . 16
5.4 The Exact Turn/River Enumeration and Next-Card Vulnerability Process . . . 17

6 Process IV: The Betting State Machine and Turn-Order Process **18**
6.1 Player Chip Accounting and the TDA Rule 19 Dead-Button Blind Process (STARTHAND) 18
6.2 The Turn-Order and Round-Completion Process (FINDNEXTACTOR) . . . 19
6.3 The Action Application and Cumulative Short All-In Reopening Process . . . 20

7 Process V: Uncalled Bet Refund, Side-Pot Slicing, and Showdown Settlement **20**
7.1 The Uncalled Bet Refund Process (REFUNDUNCALLED) . . . 20
7.2 The Horizontal Side-Pot Slicing Process (COMPUTEPOTS) . . . 21
7.3 The Showdown Evaluation, Reveal/Muck Order, and Odd-Chip Settlement Process (AWARDPOTS) . . . 22

8 Process VI: Strategic Decision Policy, Nash Push/Fold, Tournament ICM, and Solver Processes **23**

8.1 Player Archetypes and Difficulty Interpolation 23

8.2 The Preflop Heuristic Valuation Process (CHENSCORE) 23

8.3 The Short-Stack Nash Push/Fold Equilibrium Process (PUSHFOLD) 24

8.4 The Malmuth–Harville Tournament ICM Process (COMPUTEICM) 25

8.5 The Preflop and Postflop Action Selection Processes 26

8.6 Processes in Associated Full-Game GTO Solvers (CFR, Abstraction, and Subgame Solving) 27

9 Process VII: Bayesian Opponent Profiling and Hindsight-Free Self-Review **28**

9.1 The Beta-Binomial Updating Process and Two-Sigma Gate 28

9.2 The Hindsight-Free Post-Showdown Self-Review Process (REVIEWHAND) 29

10 System Invariants, Process Complexity, and Worked Walkthrough **30**

10.1 The Seven Formal System Invariants 30

10.2 Summary of Process Complexity and Runtime Benchmarks 31

10.3 Complete End-to-End Numerical Walkthrough of a 4-Player Hand Across All Seven Processes 31

1 Introduction to Poker, Probability, and Game Theory

1.1 What Is a Poker Engine? Our Overview of the Core Processes

When we build a poker engine from scratch, we are constructing a deterministic system of mathematical and logical processes by which a card game is governed from the moment a deck is shuffled to the moment the final chip is awarded. Unlike chess or Go—where the entire board is seen by both players and no randomness occurs—poker is a **stochastic game of imperfect information**. Decisions must be made under two simultaneous sources of uncertainty:

1. **Aleatoric uncertainty (chance):** Which community cards will be drawn from the remaining deck on future betting rounds?
2. **Epistemic uncertainty (hidden information):** Which two private cards are being held by each opponent, and what is revealed about those cards by their betting actions?

To ensure that rules enforcement and strategic decision-making are kept modular and free of ambiguity, our No-Limit Hold'em engine is divided into **seven distinct, sequential processes**:

- **Process I — Card Encoding, Cryptographic Shuffling, and Enclave Sealing:** Physical cards are mapped to compact integers $0 \ldots 51$, an unbiased permutation of the deck is generated using rejection-sampled cryptographic randomness, and undealt cards are isolated inside a closure-sealed dealer vault with a salted SHA-256 commitment seal.
- **Process II — Hand Evaluation and Ranking:** Any 5-, 6-, or 7-card collection is mapped into a single strictly ordered integer score so that comparing any two hands is reduced to a single integer comparison ($V(A) > V(B)$).
- **Process III — Probability, Equity, Multi-Way Bunching, and Runout Simulation:** A player's expected share of the pot (*equity*) is estimated against uniform random hands or narrowed multi-way opponent ranges (accounting for preflop card-removal bunching and board-lock split pots) using either Monte Carlo simulation or exact combinatorial enumeration.
- **Process IV — Betting State-Machine Progression (4–8 Player Tables):** Starting tables of $4, 5, 6, 7$, or 8 players are managed, the dealer button and forced blinds are advanced under Poker TDA Rule 19 dead-button rules, turn order is resolved, and cumulative short all-in reopening is enforced.
- **Process V — Pot Decomposition and Showdown Settlement:** Uncalled overbets are refunded at the end of every street and hand, unequal multi-way all-in commitments are sliced into horizontal main and side pots, eligible hands are revealed in official showdown order, and indivisible odd chips are distributed clockwise.
- **Process VI — Strategic Decision Policy, Equilibrium Solving, and Tournament ICM:** Preflop heuristic scoring is combined with our precomputed 169×19 heads-up Nash push/fold equilibrium table, action-conditioned postflop range floors, bet-pressure discounting, Malmuth–Harville tournament ICM bubble factors, and Counterfactual Regret Minimization (CFR).
- **Process VII — Bayesian Opponent Profiling and Hindsight-Free Self-Review:** Opponent betting frequencies are tracked via Beta-Binomial conjugate priors gated by a 2σ credible interval, and post-decision regret buckets are updated after showdown against revealed hands without ever peeking at mucked cards.

1.2 Introduction to Probability from Scratch

Every calculation performed inside our engine is grounded in four foundational concepts from probability theory:

Definition 1.1 (Sample Space, Events, and Uniform Measure)**.** A **sample space** Ω is defined as the set of all possible elementary outcomes of a random process. When cards are dealt fairly from a finite deck, every outcome $\omega \in \Omega$ is equally likely, so the probability of any event $E \subseteq \Omega$ is given by the ratio of favourable outcomes to total possible outcomes:

$$\mathbb{P}(E) = \frac{|E|}{|\Omega|}. \tag{1}$$

Definition 1.2 (Combinations and the Binomial Coefficient)**.** Because the order in which cards are dealt into a hand or onto a board does not alter the resulting poker hand, unordered subsets are counted. The number of ways in which k distinct cards can be chosen from a pool of n remaining cards is given by the **binomial coefficient** (read "n choose k"):

$$\binom{n}{k} = \frac{n!}{k!\,(n-k)!} = \frac{n(n-1)\cdots(n-k+1)}{k(k-1)\cdots 1}. \tag{2}$$

For example, the number of distinct 2-card starting hands from a 52-card deck is $\binom{52}{2} = \frac{52\times 51}{2} = 1,326$, and the number of distinct 5-card hands is $\binom{52}{5} = \frac{52\times 51\times 50\times 49\times 48}{5\times 4\times 3\times 2\times 1} = 2,598,960$.

Definition 1.3 (Conditional Probability and Card Removal)**.** Whenever cards are revealed at the table (such as our own 2 hole cards and b community cards on the board), the sample space is reduced from 52 cards to the $52-2-b$ unseen cards. The **conditional probability** of event A given observed information B is written as:

$$\mathbb{P}(A \mid B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)}. \tag{3}$$

In poker, conditioning on known cards is referred to as **card removal** (or *blocker effects*), whereas conditioning on an opponent's betting actions—by which strong hands are made more likely and weak hands less likely—is referred to as **Bayesian range narrowing**.

Definition 1.4 (Random Variables and Expected Value (EV))**.** A **random variable** X assigns a numerical payoff (measured in chips, big blinds, or tournament prize share) to each possible outcome of a hand. Its **expected value** $\mathbb{E}[X]$ (or EV) is defined as the probability-weighted average of all possible payoffs:

$$\mathrm{EV} = \mathbb{E}[X] = \sum_{\omega\in\Omega} \mathbb{P}(\omega)\, X(\omega). \tag{4}$$

When a decision has $\mathrm{EV} > 0$ (denoted $+$EV), a long-run profit is produced when that decision is repeated over thousands of hands; when $\mathrm{EV} < 0$ ($-$EV), a long-run loss is incurred, regardless of whether any single hand happens to be won or lost.

1.3 Introduction to Game Theory from Scratch

While probability tells us how likely a hand is to improve against a fixed set of cards, **game theory** is used to study how rational decision-makers interact when each player's payoff depends on the strategies chosen by others.

Plain-English Intuition: Why We Need Game Theory in Our Poker Engine. Suppose our engine only ever bet when it held an unbeatable hand and always checked when it held a weak hand. An observant opponent would simply fold every time our engine bet and win the pot every time our engine checked. Conversely, if our engine bluffed on every hand, our bets would be called down and huge pots would be lost. Game theory allows us to calculate the exact mathematical mix of **value bets** (betting strong hands for profit), **bluffs** (betting weak hands to force folds), and **defensive calls** so that our strategy cannot be exploited no matter how our opponents adjust.

Three foundational principles of game theory are used throughout our engine and solver architecture:

1. **Pure vs. Mixed Strategies:** Under a *pure strategy*, the exact same action is always taken in a given situation (e.g., always raising with A♠ A♡). Under a *mixed strategy* $\sigma(I)$, a probability distribution is assigned over legal actions at an information state I (e.g., raising 70% of the time and calling 30% of the time) so that our exact holding cannot be deduced from our action.

2. **Nash Equilibrium (Game-Theoretic Optimal / GTO Play):** In a two-player zero-sum game (where every chip won by Player 1 is lost by Player 2), a strategy pair $(\sigma_1^\star, \sigma_2^\star)$ is a **Nash equilibrium** if neither player's expected value can be increased by unilaterally changing their strategy:
$$u_1(\sigma_1^\star, \sigma_2^\star) \geq u_1(\sigma_1, \sigma_2^\star) \quad \forall \sigma_1, \qquad u_2(\sigma_1^\star, \sigma_2^\star) \geq u_2(\sigma_1^\star, \sigma_2) \quad \forall \sigma_2. \tag{5}$$
When a Nash equilibrium is played, our strategy is guaranteed to be **unexploitable**: even an omniscient opponent who knows our exact strategy probabilities cannot achieve positive expected value against us over the long run (once seat positions are alternated).

3. **Minimum Defense Frequency (MDF) and Bluff-to-Value Ratio:** Suppose B chips are bet by an aggressor into a pot of P_0 chips on the river. If the aggressor is bluffing, B chips are risked to win P_0 chips. For a bluff to break even (EV $= 0$), the defender must fold with probability $f^\star = \frac{B}{P_0+B}$. Therefore, to prevent the aggressor from profiting with any two random cards as a bluff, the defender must call or raise with at least the **Minimum Defense Frequency**:
$$\text{MDF} = 1 - f^\star = 1 - \frac{B}{P_0 + B} = \frac{P_0}{P_0 + B}. \tag{6}$$
Conversely, a defender facing a bet of B into a pot of P_0 must call B to contest a final pot of $P_0 + 2B$, receiving pot odds of $\frac{B}{P_0+2B}$. To make the defender indifferent between calling and folding with a bluff-catcher, bluffs and value hands must be placed into the aggressor's betting range in the ratio Bluffs : Value $= B : (P_0 + B)$, so the optimal fraction of bluffs inside a polarized river bet is:
$$\alpha^\star_{\text{bluff}} = \frac{B}{P_0 + 2B}. \tag{7}$$
For a pot-sized bet ($B = P_0$), MDF $= \frac{1}{2} = 50\%$ of the defender's range must be defended, and every 2 value bets should be balanced with 1 bluff ($\alpha^\star_{\text{bluff}} = \frac{1}{3} \approx 33.3\%$).

2 Rules, Hand Rankings, Pot Size, and Core Terminologies

2.1 The Deck, 4–8 Player Positions, Blinds, and Street Progression

In our engine, standard No-Limit Texas Hold'em is played with a 52-card French deck comprising 13 **ranks** in each of 4 **suits** (♠ Spades, ♡ Hearts, ♢ Diamonds, ♣ Clubs). Ranks are ordered from lowest to highest as:

$$2 < 3 < 4 < 5 < 6 < 7 < 8 < 9 < 10\ (\text{T}) < \text{J}\ (\text{Jack}) < \text{Q}\ (\text{Queen}) < \text{K}\ (\text{King}) < \text{A}\ (\text{Ace}),$$

where the Ace (A) can also serve as the lowest card (1) to complete a five-high straight (A–2–3–4–5, called the **wheel**). No relative ranking is assigned to suits in Hold'em: A♠ K♠ Q♠ J♠ T♠ and A♡ K♡ Q♡ J♡ T♡ tie and split the pot.

When we designed the table geometry, we configured starting tables to support **4, 5, 6, 7, or 8 players** (`count` $\in \{4, 5, 6, 7, 8\}$), with seats arranged clockwise around an elliptical felt rail and adaptive seat-plate scaling for 7-max and 8-max full-ring play. As players bust out during a game, the active lineup naturally contracts down to 3-handed and 2-handed (heads-up) play.

Before each hand begins, one seat is marked with the **dealer button (BTN)**, and forced bets are posted by the blind seats before any cards are dealt:

- **Small Blind (SB):** Posted by the seat immediately clockwise from the button, normally equal to half of the big blind ($\texttt{sb} = \lfloor \texttt{bb}/2 \rfloor$).
- **Big Blind (BB):** Posted by the seat clockwise from the small blind, equal to the full minimum bet unit (bb) of the table.
- **Position Badges Across 4–8 Player Tables:** Walking clockwise from the big blind to the button, active non-blind seats are badged as UTG (4-handed), UTG, CO (5-handed), UTG, HJ, CO (6-handed), UTG, UTG + 1, HJ, CO (7-handed), and UTG, UTG + 1, MP, HJ, CO (8-handed).
- **Heads-Up Exception (When Only 2 Funded Players Remain):** Once eliminations reduce the table to two active players, the button posts the small blind (SB · BTN) and acts *first* preflop, while the big blind acts *first* on all postflop streets.

A single hand progresses through up to four betting rounds (**streets**), followed by a **showdown** if more than one player remains un-folded:

1. **Preflop (0 board cards):** Two private face-down **hole cards** are dealt clockwise to each funded player starting from the small blind seat. Betting begins with the player immediately clockwise from the big blind (UTG) and proceeds clockwise around the table.
2. **Flop (3 board cards):** One top card is discarded face down (**burn card**) and 3 shared community cards are dealt face up to the centre of the table (**the board**). Betting begins with the first active player clockwise from the button.
3. **Turn (4 board cards):** A second burn card is discarded and a 4th community card is dealt face up, followed by a third round of betting.
4. **River (5 board cards):** A third burn card is discarded and a 5th and final community card is dealt face up, followed by the final round of betting.
5. **Showdown:** Remaining un-folded players reveal their hole cards in official showdown order (starting with the last river aggressor, or clockwise from left of the button if the river was checked through). Each player forms the single highest-ranking 5-card poker hand from the 7 cards available to them (their 2 hole cards plus the 5 community cards), while losing AI hands that are not all-in may muck face-down.

2.2 Legal Betting Actions and Raising Rules

When the action reaches a player on any street, let currentBet denote the highest street bet placed so far on that street, let $p.\texttt{bet}$ denote the amount that player p has already put in on the current street, and let $\texttt{toCall} = \max(0, \texttt{currentBet} - p.\texttt{bet})$ denote the additional chips required to stay in the hand. Depending on toCall and the player's remaining chip stack, up to five actions are permitted:

- **Fold:** The hand is forfeited immediately. Previously committed chips remain in the pot, and no portion of the pot can be won by that player.
- **Check:** The action is passed without wagering additional chips. Legal *if and only if* $\texttt{toCall} = 0$.
- **Call:** $\min(p.\texttt{stack}, \texttt{toCall})$ chips are committed to match the current street bet. If $p.\texttt{stack} \leq \texttt{toCall}$, the call is an **all-in call** for the player's entire remaining stack.
- **Bet / Raise:** The street's wager is increased to a target total $\tau > \texttt{currentBet}$ (called a *bet* when $\texttt{currentBet} = 0$, and a *raise* when $\texttt{currentBet} > 0$). In No-Limit Hold'em, a full raise must increase currentBet by at least the size of the previous largest full bet or raise increment on that street (lastRaise, initialized to bb):

$$\tau_{\min} = \max\big(\texttt{currentBet} + \texttt{lastRaise}, \texttt{bb}\big), \qquad \tau_{\max} = p.\texttt{bet} + p.\texttt{stack}. \tag{8}$$

- **All-In:** The entirety of one's remaining stack is wagered ($\tau = p.\texttt{bet} + p.\texttt{stack}$). If an all-in wager exceeds `currentBet` by less than `lastRaise` ($\tau - \texttt{currentBet} < \texttt{lastRaise}$), it is treated as a **short all-in**: it is always legal for the all-in player, but it does *not* by itself reopen the right to re-raise for players who have already acted since the last full raise.

2.3 Complete Five-Card Hand Rankings and Combinatorial Frequencies

At showdown, every five-card poker hand is classified into one of ten mutually exclusive **categories** $\kappa \in \{0, 1, \ldots, 9\}$. Within any single category, ties are broken by comparing an ordered tuple of up to five **tie-breaker ranks** (the primary pattern ranks followed by the remaining side cards, called **kickers**) from most significant to least significant.

κ	Category Name	Exact Count	Probability	Combinatorial Derivation & Tie-Break Order $(k_0, k_1, \ldots)$
9	**Royal Flush**	4	0.000154%	$\binom{4}{1} = 4$ Ace-high straight flushes (A–K–Q–J–T of one suit). Tie-breaker: (12).
8	**Straight Flush**	36	0.001385%	$9 \times \binom{4}{1} = 36$ non-royal 5-card sequences in one suit (5-high wheel through K-high). Tie-breaker: (s), where s is the top rank.
7	**Four of a Kind**	624	0.024010%	$\binom{13}{1}\binom{4}{4}\binom{12}{1}\binom{4}{1} = 13 \times 48 = 624$. Four cards of one rank q plus one kicker k. Tie-breaker: (q, k).
6	**Full House**	3,744	0.144058%	$\binom{13}{1}\binom{4}{3}\binom{12}{1}\binom{4}{2} = 13 \times 4 \times 12 \times 6 = 3,744$. Three of a kind t plus a pair p. Tie-breaker: (t, p).
5	**Flush**	5,108	0.196540%	$\binom{4}{1}\binom{13}{5} - 40 = 4(1,287) - 40 = 5,108$. Any 5 cards of the same suit, not in sequence. Tie-breaker: $(r_0, r_1, r_2, r_3, r_4)$.
4	**Straight**	10,200	0.392465%	$10 \times (4^5 - 4) = 10 \times 1,020 = 10,200$. Five consecutive ranks across mixed suits. Tie-breaker: (s).
3	**Three of a Kind**	54,912	2.112845%	$\binom{13}{1}\binom{4}{3}\binom{12}{2}\binom{4}{1}^2 = 13 \times 4 \times 66 \times 16 = 54,912$. Trips t plus two kickers $k_1 > k_2$. Tie-breaker: (t, k_1, k_2).
2	**Two Pair**	123,552	4.753902%	$\binom{13}{2}\binom{4}{2}^2\binom{11}{1}\binom{4}{1} = 78 \times 36 \times 44 = 123,552$. High pair p_1, low pair p_2, kicker k. Tie-breaker: (p_1, p_2, k).
1	**One Pair**	1,098,240	42.256903%	$\binom{13}{1}\binom{4}{2}\binom{12}{3}\binom{4}{1}^3 = 13 \times 6 \times 220 \times 64 = 1,098,240$. Pair p plus kickers $k_1 > k_2 > k_3$. Tie-breaker: (p, k_1, k_2, k_3).
0	**High Card**	1,302,540	50.117739%	$\left(\binom{13}{5} - 10\right)(4^5 - 4) = 1,277 \times 1,020 = 1,302,540$. Five unpaired, non-sequential, non-suited ranks: $(r_0, r_1, r_2, r_3, r_4)$.
	Total Hands	**2,598,960**	**100.000000%**	$\binom{52}{5} = 2,598,960$ distinct 5-card subsets (7,462 equivalence classes).

Table 1: Complete combinatorial breakdown of all $\binom{52}{5} = 2,598,960$ five-card hands in Texas Hold'em.

Remark 2.1 (Note on Royal Flush vs. Straight Flush Categorisation). Inside our mathematical evaluation algorithms, a Royal Flush (A–K–Q–J–T suited) is naturally represented as the highest-ranking Straight Flush ($\kappa = 8$ with top rank $s = 12$). Whether it is separated as $\kappa = 9$ for human display or kept as $\kappa = 8, s = 12$ inside our evaluator, the exact same strict ordering is preserved over all 2,598,960 hands.

2.4 Pot Size, Pot Odds, Stack-to-Pot Ratio, and Side Pots

To translate chip commitments into mathematical incentives inside our engine, four quantitative definitions are used:

1. **Total Pot Size (P):** At any decision point, the total pot P is defined as the sum of all chips committed across prior and current streets by all players (including players who have since folded). When we face a call cost of $C = \texttt{toCall}$ into a pot of size P (where P already includes the opponent's bet), calling C contests a final pot of $P + C$.

2. **Pot Odds and Break-Even Equity ($q^\star$):** Suppose C chips are called into a current pot P, and no further betting occurs on later streets. If our probability of winning (or expected pot share) is $q \in [0, 1]$, our expected value of calling is:

$$\mathrm{EV}(\textsc{Call}) = q(P + C) - C = qP - (1 - q)C. \tag{9}$$

   Since folding always yields $\mathrm{EV}(\textsc{Fold}) = 0$, calling is profitable ($\mathrm{EV}(\textsc{Call}) \geq 0$) if and only if our equity q meets or exceeds the **pot odds threshold** $q^\star$:

$$q \geq q^\star = \texttt{potOdds} = \frac{C}{P + C}. \tag{10}$$

   For example, if the pot before an opponent's bet is 100 and 50 is bet by the opponent, the pot is now $P = 150$ and our cost to call is $C = 50$. Our pot odds are $\frac{50}{150+50} = \frac{50}{200} = 25.0\%$: winning more than 1 time in 4 makes the call profitable.

3. **Effective Stack (S_{eff}) and Stack-to-Pot Ratio (SPR):** A player with $5,000$ chips cannot win $5,000$ chips from an opponent who only has 400 chips. In a heads-up confrontation between players A and B, the **effective stack** is $S_{\text{eff}} = \min(\texttt{stack}_A, \texttt{stack}_B)$. At the start of the flop, the **Stack-to-Pot Ratio** is defined as:

$$\mathrm{SPR} = \frac{S_{\text{eff}}}{P_{\text{flop}}}. \tag{11}$$

   When SPR is low ($\mathrm{SPR} \leq 3$), the entire remaining stack is committed by a single pot-sized bet and call, which favours high pairs (A A, K K, Q Q) and top pair; when SPR is high ($\mathrm{SPR} \geq 10$), three full streets of geometric betting can be played, which favours suited connectors and small pairs that can make disguised straights, flushes, or full houses (**implied odds**).

4. **Main Pot and Side Pots:** When one or more players go all-in with smaller stacks than remaining active players, the total pot is sliced horizontally by commitment depth. The **main pot** is the lowest slice contested by all non-folded players; each subsequent **side pot** contains the excess chips matched only by the deeper-stacked players and can only be won by those deeper-stacked players.

2.5 Comprehensive Terminology Reference

In Table 2, we summarise the core operational, probabilistic, and game-theoretic terms that are used throughout our seven engine processes.

3 Process I: Card Representation, Cryptographic Shuffling, and Enclave Sealing

3.1 The Bijective Integer Encoding Process

In the first process of our engine, the 52 cards of the deck are represented as primitive integers so that extracting a card's rank or suit requires zero memory allocations and zero string comparisons.

Term	Domain	First-Principles Definition
Hole Cards (H)	Rules	The two private face-down cards dealt to each player preflop.
Board (B)	Rules	Up to five shared face-up community cards dealt across the flop (3), turn (4), and river (5).
Kicker	Ranking	An unpaired side card used to break ties between hands of the same category and primary rank (e.g., A♠ K♢ beats A♡ Q♣ on A♣ 8♠ 4♡ 2♢ 3♣ via the King kicker).
Outs	Probability	Unseen cards remaining in the stub that will improve a drawing hand to a likely winning hand on the next street.
Equity (q or e)	Probability	A player's expected fraction of the pot ($q \in [0,1]$) averaged over all possible remaining board runouts and opponent holdings, counting an s-way tie as $1/s$ of a win.
Bunching	Probability	The skew toward high cards in the remaining stub caused when multiple early-position players fold low-scoring hands preflop on a 4–8 player table.
Range ($\mathcal{R}$)	Game Theory	The probability distribution over all 1,326 two-card holdings that a player could plausibly hold given their actions so far in the hand.
VPIP / PFR	Profiling	*Voluntarily Put Money in Pot* (percentage of hands where a player calls or raises preflop) and *Preflop Raise* (percentage of hands raised preflop).
Dead Button	State Machine	Under Poker TDA Rule 19, advancing the blinds strictly forward so the small blind or button may temporarily sit on an empty busted seat rather than letting a player skip a blind.
Bubble Factor (BF)	Tournaments	The ratio of tournament prize equity lost upon busting to prize equity gained upon winning a pot, divided by chip odds ($BF \geq 1.0$).
Counterfactual Regret	Solver Math	How much more expected value a player would have achieved at an information set had a specific action been chosen instead of the current mixed strategy.

Table 2: Foundational poker, probability, and game-theory terminology.

Every card is encoded as a single integer $c \in \{0, 1, \ldots, 51\}$ via the bijection:

$$c = 4\rho + \sigma, \qquad \rho(c) = \lfloor c/4 \rfloor = c \gg 2, \qquad \sigma(c) = c \bmod 4 = c \,\&\, 3, \tag{12}$$

where $\rho \in \{0, 1, \ldots, 12\}$ is the zero-indexed rank ($0 \mapsto 2, 1 \mapsto 3, \ldots, 8 \mapsto \mathrm{T}, 9 \mapsto \mathrm{J}, 10 \mapsto \mathrm{Q}, 11 \mapsto \mathrm{K}, 12 \mapsto \mathrm{A}$) and $\sigma \in \{0, 1, 2, 3\}$ is the suit index ($0 \mapsto$ ♠, $1 \mapsto$ ♡, $2 \mapsto$ ♢, $3 \mapsto$ ♣).

Plain-English Intuition: Why We Replace Division with Bitwise Shifts. In binary (base 2), when an integer ρ is multiplied by 4, its bits are shifted two places to the left ($\rho \ll 2$), leaving the lowest two bits empty to store the suit $\sigma \in \{0, 1, 2, 3\}$ (which in binary is $00_2, 01_2, 10_2, 11_2$). To recover the rank ρ, shifting right by two bits ($c \gg 2$) drops the suit bits in a single CPU cycle. To recover the suit σ, bitwise AND with 3 ($c \,\&\, 3$, where $3 = 11_2$) masks out everything except the lowest two bits. Furthermore, $1 \ll \rho$ creates a 13-bit rank flag where bit ρ is set to 1, allowing an entire 7-card hand's ranks to be combined via bitwise OR.

3.2 The Unbiased Cryptographic Shuffling Process

A full 52-card deck has $52! \approx 8.0658 \times 10^{67}$ possible orderings. Since $\log_2(52!) \approx 225.58$ bits, only an infinitesimal fraction of all legal decks could ever be reached by a standard 32-bit or 64-bit pseudo-random number generator. Moreover, when a raw 32-bit random integer $X \in \{0, \ldots, 2^{32} - 1\}$ is reduced modulo n ($X \bmod n$), **modulo bias** is introduced whenever n does not divide 2^{32}, because the remainder $r = 2^{32} \bmod n$ causes the first r outcomes to have one extra preimage.

In our shuffling process, both weaknesses are eliminated by pairing a hardware-seeded cryptographic random source with **rejection sampling** (Algorithm 1).

Algorithm 1: CRYPTOGRAPHICSHUFFLE — unbiased rejection sampling and in-place Fisher–Yates permutation

Input: deck array $D[0 \dots 51]$; integer bound $n \geq 1$
Output: uniformly permuted deck D over all 52! orderings

Procedure SECURERANDOMINT(n):
  if $n \leq 1$ **then return** 0
  $L \leftarrow \lfloor 2^{32}/n \rfloor \cdot n$ ▷ *largest multiple of n within* 2^{32}
  repeat
    Draw 32-bit cryptographic random integer $X \sim \text{Uniform}(\{0, 1, \dots, 2^{32} - 1\})$
  until $X < L$ ▷ *reject the incomplete tail* $[L, 2^{32})$
  return $X \bmod n$
Procedure SHUFFLEDECKINPLACE(D):
  for $i \leftarrow 51$ **downto** 1 **do**
    $j \leftarrow$ SECURERANDOMINT($i + 1$) ▷ *uniform index in* $\{0, \dots, i\}$
    Swap $D[i] \leftrightarrow D[j]$
  return D

Proposition 3.1 (Exact Uniformity of the Rejection-Sampled Shuffle Process)**.** *For every* $n \in \{1, \dots, 52\}$*, each integer* $k \in \{0, \dots, n-1\}$ *is returned by* SECURERANDOMINT(n) *with probability exactly* $1/n$ *in at most* $1 + 52/2^{32} < 1.00000002$ *expected draws, and each of the* 52! *deck permutations is produced by* SHUFFLEDECKINPLACE *with probability exactly* $1/52!$*.*

Proof. Because $L = \lfloor 2^{32}/n \rfloor \cdot n$ is an exact multiple of n, the acceptance region $\{0, 1, \dots, L-1\}$ is partitioned into n residue classes modulo n, each of identical cardinality $M = \lfloor 2^{32}/n \rfloor$. Conditioning on $X < L$, every residue $k \in \{0, \dots, n-1\}$ is chosen with probability $M/(nM) = 1/n$. In the Fisher–Yates loop, step i selects among $i + 1$ remaining elements uniformly and independently for $i = 51, 50, \dots, 1$, generating $\prod_{i=1}^{51}(i+1) = 52!$ distinct execution paths, each with probability $1/52!$. □

3.3 The Closure-Sealed Dealer Enclave and Provably Fair Commitment Process (DEALERVAULT)

When a client-side poker engine stores the undealt deck stub or opponent hole cards on a global state object, two security pitfalls arise: first, any user opening the browser console can inspect future community cards or hidden AI holdings; second, if any helper routine accidentally mutates a player's hole-card array, the 52-card bijection is silently corrupted.

To bring our engine as close as possible to a production online server, the shuffled deck and all private hole cards are encapsulated inside a closure-isolated enclave (DEALERVAULT, Algorithm 2). Before the first card of a hand is dealt, a 128-bit cryptographic salt is generated and a **Provably Fair SHA-256 commitment seal** is published over the full 52-card permutation and salt:

$$\texttt{seal} = \text{SHA256}\big(\text{encodeDeck}(D_0) \,\|\, \text{“|”} \,\|\, \texttt{salt}\big)_{0..31}. \tag{13}$$

On the public table state, unrevealed AI seats store only a masked placeholder $(-1, -1)$ until their cards are legally tabled at showdown, and at the end of every hand the initial deck order and salt are verified against `seal`.

Algorithm 2: DEALERVAULT — closure-sealed card enclave and Provably Fair SHA-256 commitment seal

Input: funded seat list E; small blind seat `sbSeat`; table size N_p

Output: published commitment hash `seal` and isolated card-access closures

Enclave State (Private Closure Variables): *deck*$[0\ldots51]$, *holeStore*$[\cdot]$, *burned*$[\cdot]$, *salt*, *seal*, *initialOrder*

Procedure INITHAND(E, `sbSeat`, N_p):

  deck $\leftarrow$ SHUFFLEDECKINPLACE($[0,1,\ldots,51]$); *initialOrder* $\leftarrow$ copy(*deck*)

  Draw 128-bit cryptographic hex salt *salt*; compute *seal* via Eq. (13)

  for $r \leftarrow 1$ **to** 2 **do for** $i \leftarrow 0$ **to** $N_p - 1$ **do**

    $s \leftarrow (\texttt{sbSeat} + i) \bmod N_p$; **if** $s \in E$ **then** append pop(*deck*) to *holeStore*$[s]$

  return *seal* ▷ *AI seats on public state receive masked* $[-1,-1]$

Function DEALSTREET(k):

  Append pop(*deck*) to *burned*; **return** [pop(*deck*) for $1\ldots k$]

Function VERIFYSEAL():

  Recompute *seal*$'$ from *initialOrder* and *salt* via Eq. (13); **return** (*seal*$'$ = *seal*)

4 Process II: Hand Evaluation and Associated Evaluator Architectures

4.1 The Base-13 Lexicographical Packing Process

During a hand evaluation or Monte Carlo simulation, millions of five-card and seven-card hands must be compared per second. When two hands are compared via multi-branch conditional logic (first checking the category κ, then checking the first tie-breaker k_0, then k_1, and so on), severe CPU branch-prediction penalties are incurred.

To eliminate branching during comparisons, a hand's category $\kappa \in \{0,\ldots,8\}$ and its tie-breaker tuple $k = (k_0,\ldots,k_{m-1}) \in \{0,\ldots,12\}^m$ (with $m \le 5$, zero-padded on the right to length 5) are packed into a single integer via a **6-digit base-13 positional number** (Algorithm 3):

$$V(\kappa,k) = \kappa \cdot 13^5 + \sum_{i=0}^{4} k_i \cdot 13^{4-i}, \qquad \text{where } k_i = 0 \text{ for } i \ge m, \text{ and } 13^5 = 371,293. \tag{14}$$

Algorithm 3: ENCODE — pack category and tie-breaker ranks into a single base-13 integer

Input: category $\kappa \in \{0,\ldots,8\}$; tie-breaker tuple $k = (k_0,\ldots,k_{m-1})$ with $m \le 5$

Output: strictly ordered integer score $V \in [0,\ 3,341,636]$

$v \leftarrow \kappa$

for $i \leftarrow 0$ **to** 4 **do**

  $v \leftarrow 13 \cdot v + (k_i \text{ if } i < m \text{ else } 0)$

return v

Proposition 4.1 (Order Isomorphism of Base-13 Score Packing)**.** *For any two five-card hands H and H' with category/tie-breaker representations (κ,k) and (κ',k'), $V(\kappa,k) > V(\kappa',k')$ if and only if $(\kappa,k) \succ_{\text{lex}} (\kappa',k')$ under the official rules of Texas Hold'em. Moreover, comparisons are never altered by zero-padding tuples of length $m < 5$.*

Proof. Let $(d_0,d_1,d_2,d_3,d_4,d_5) = (\kappa,k_0,k_1,k_2,k_3,k_4) \in \{0,\ldots,12\}^6$. At the first index $j \in \{0,\ldots,5\}$ where $d_j \ne d'_j$, suppose $d_j > d'_j$, so $d_j - d'_j \ge 1$. The maximum possible countervailing contribution

from all less-significant positions $i > j$ is:

$$\sum_{i=j+1}^{5} 12 \cdot 13^{5-i} = 12 \cdot \frac{13^{5-j}-1}{13-1} = 13^{5-j} - 1 < 1 \cdot 13^{5-j}.$$

Thus $V(\kappa,k) - V(\kappa',k') \geq 13^{5-j} - (13^{5-j}-1) = 1 > 0$. Finally, two hands of the same category κ always share the exact same unpadded tuple length $m(\kappa) \in \{1,2,3,4,5\}$, so identical trailing zeros are placed in positions $m(\kappa)\ldots 4$; hands of different categories $\kappa \neq \kappa'$ are already separated at $j = 0$ by at least $13^5 = 371,293$. □

4.2 The Five-Card Classification Process (EVAL5)

Given an exact 5-card subset $C = (c_0, c_1, c_2, c_3, c_4)$, our five-card classification process (Algorithm 4) builds a 13-element rank histogram $cnt[0\ldots12]$, sorts the distinct present ranks into a descending list U, groups each rank $r \in U$ into a bucket $G[cnt[r]]$ by its multiplicity (1, 2, 3, or 4), and tests for flushes and straights.

Algorithm 4: EVAL5 — classify and score a 5-card hand from rank multiplicities and suit parity

Input: five distinct cards $C = (c_0, c_1, c_2, c_3, c_4)$
Output: base-13 score $V(C) = \text{ENCODE}(\kappa, k)$

1 Extract ranks $R \leftarrow (\rho(c_0), \ldots, \rho(c_4))$ and suits $S \leftarrow (\sigma(c_0), \ldots, \sigma(c_4))$
2 *flush* $\leftarrow (S_0 = S_1 = S_2 = S_3 = S_4)$
3 Build multiplicity histogram $cnt[r] \leftarrow |\{i : R_i = r\}|$ for $r \in \{0, \ldots, 12\}$
4 $U \leftarrow$ distinct ranks with $cnt[r] > 0$, sorted in strictly descending order
5 Partition U by multiplicity into lists $G[1], G[2], G[3], G[4]$ (each inheriting descending order)
6 *straight* $\leftarrow$ **false**; $s \leftarrow -1$
7 **if** $|U| = 5$ **then**
8 | **if** $U_0 - U_4 = 4$ **then** *straight* $\leftarrow$ **true**; $s \leftarrow U_0$ ▷ *standard consecutive straight*
9 | **else if** $U = (12,3,2,1,0)$ **then** *straight* $\leftarrow$ **true**; $s \leftarrow 3$ ▷ *wheel straight A-2-3-4-5*
10 **if** *straight* $\wedge$ *flush* **then return** ENCODE$(8, (s))$
11 **if** $|G[4]| = 1$ **then return** ENCODE$(7, (G[4]_0, G[1]_0))$
12 **if** $|G[3]| = 1 \wedge |G[2]| = 1$ **then return** ENCODE$(6, (G[3]_0, G[2]_0))$
13 **if** *flush* **then return** ENCODE$(5, U)$
14 **if** *straight* **then return** ENCODE$(4, (s))$
15 **if** $|G[3]| = 1$ **then return** ENCODE$(3, (G[3]_0, G[1]_0, G[1]_1))$
16 **if** $|G[2]| = 2$ **then return** ENCODE$(2, (G[2]_0, G[2]_1, G[1]_0))$
17 **if** $|G[2]| = 1$ **then return** ENCODE$(1, (G[2]_0, G[1]_0, G[1]_1, G[1]_2))$
18 **return** ENCODE$(0, U)$

Proposition 4.2 (Span-Contiguity Straight Detection and Wheel Rank Assignment)**.** *Let $U = (U_0 > U_1 > U_2 > U_3 > U_4)$ be five strictly decreasing integers. Then $U_0 - U_4 = 4$ if and only if $U = (U_0, U_0-1, U_0-2, U_0-3, U_0-4)$. Furthermore, when $s = 3$ (rank 5) is assigned to the Ace-low wheel $U = (12,3,2,1,0)$, it is placed strictly below the 6-high straight ($s = 4$) and $(12,3,2,1,0)$ can never be reached as a High-Card hand ($\kappa = 0$) or ordinary Flush ($\kappa = 5$).*

Proof. Since the U_i are strictly decreasing integers, each consecutive difference satisfies $U_i - U_{i+1} \geq 1$ for $i = 0,1,2,3$. When the four differences are summed, $U_0 - U_4 = \sum_{i=0}^{3}(U_i - U_{i+1}) \geq 4$ is obtained, with equality if and only if every step equals 1. For the wheel $U = (12,3,2,1,0)$, the hand is routed by Lines 9–10 and 14 to $\kappa = 8$ (if suited) or $\kappa = 4$ (if unsuited) with $s = 3 < 4$, ensuring 5-high straight $<$ 6-high straight. □

4.3 The 21-Subset Seven-Card Evaluation Process (BEST7)

At showdown, 7 cards (2 hole cards plus 5 board cards) are available to each surviving player, and the best 5-card subset is formed. Because $\binom{7}{5} = \frac{7\times6}{2} = 21$, all 21 five-card subsets are enumerated by our direct combinatorial process (Algorithm 5), EVAL5 is called on each, and both the maximum score $V^\star$ and the winning 5-card subset $C^\star$ are returned so that the winning cards can be highlighted on the felt.

Algorithm 5: BEST7 — exhaustive 21-subset 7-card evaluation returning both score and winning 5-card subset

Input: collection $A = (a_0, \ldots, a_{n-1})$ of $n \in \{5, 6, 7\}$ cards
Output: pair $(V^\star, C^\star)$ where $V^\star = \max_{C \subset A, |C|=5} \text{EVAL5}(C)$

$V^\star \leftarrow -1$; $C^\star \leftarrow$ **nil**
for $i_0 \leftarrow 0$ **to** $n-5$ **do**
  for $i_1 \leftarrow i_0+1$ **to** $n-4, \ldots, i_4 \leftarrow i_3+1$ **to** $n-1$ **do**
    $C \leftarrow (a_{i_0}, a_{i_1}, a_{i_2}, a_{i_3}, a_{i_4})$
    $v \leftarrow \text{EVAL5}(C)$
    if $v > V^\star$ **then** $V^\star \leftarrow v$; $C^\star \leftarrow C$
return $(V^\star, C^\star)$

4.4 The Single-Pass Bitmask and Lookup-Table Evaluation Process (RANK7FAST)

When I first profiled our Monte Carlo equity engine, calling EVAL5 21 times per hand inside a rollout created heavy garbage-collection pressure from temporary array allocations. To eliminate that bottleneck, we designed RANK7FAST (Algorithm 6), by which the exact same base-13 score is computed in a **single pass over the 7 cards** with zero allocations, using three precomputed 8,192-entry lookup tables indexed by any 13-bit rank mask $m \in \{0, 1, \ldots, 2^{13} - 1 = 8,191\}$:

1. `POPC`$[m]$: Population count (number of set 1-bits in m).
2. `STR8`$[m]$: Highest straight top-rank $s \in \{3, 4, \ldots, 12\}$ present in m (testing $(m \,\&\, (31 \ll r)) = (31 \ll r)$ from $r = 8$ down to 0, and checking $(m \,\&\, \texttt{0x100F}) = \texttt{0x100F}$ for the wheel A–2–3–4–5 where $2^{12} + 2^3 + 2^2 + 2^1 + 2^0 = 4111 = \texttt{0x100F}$), or -1 if no straight exists.
3. `TOP5T`$[m]$: Precomputed base-13 sum $\sum_{i=0}^{4} r_i \cdot 13^{4-i}$ of the five highest set bits $r_0 > r_1 > r_2 > r_3 > r_4$ of m.

Algorithm 6: RANK7FAST — single-pass bitmask and lookup-table 7-card evaluator

Input: array A of $n \in \{5,6,7\}$ encoded cards
Output: exact base-13 score $V(A)$ identical to BEST7(A)

$s_0, s_1, s_2, s_3 \leftarrow 0,0,0,0$; $all \leftarrow 0$; $cnt[0\ldots12] \leftarrow 0$
for $i \leftarrow 0$ **to** $n-1$ **do**
  $c \leftarrow A_i$; $r \leftarrow c \gg 2$; $u \leftarrow c \,\&\, 3$; $b \leftarrow 1 \ll r$
  $all \leftarrow all \mid b$; $cnt[r] \leftarrow cnt[r]+1$; $s_u \leftarrow s_u \mid b$
$f_m \leftarrow s_0$ if $\texttt{POPC}[s_0] \geq 5$, s_1 if $\texttt{POPC}[s_1] \geq 5$, s_2 if $\texttt{POPC}[s_2] \geq 5$, s_3 if $\texttt{POPC}[s_3] \geq 5$, else 0
if $f_m \neq 0$ **then** ▷ *at most one suit can have* ≥ 5 *of 7 cards*
  $sf \leftarrow \texttt{STR8}[f_m]$
  if $sf \geq 0$ **then return** $8 \cdot 13^5 + sf \cdot 13^4$ ▷ *Straight Flush*
  return $5 \cdot 13^5 + \texttt{TOP5T}[f_m]$ ▷ *Flush*
Scan $r = 12\ldots0$ to record top quad q_0, up to two trips $t_0 > t_1$, and up to three pairs $p_0 > p_1 > p_2$
if $q_0 \geq 0$ **then return** $7 \cdot 13^5 + q_0 \cdot 13^4 + (\max_{r \neq q_0, cnt[r]>0} r) \cdot 13^3$ ▷ *Four of a Kind*
if $t_0 \geq 0 \wedge (t_1 \geq 0 \vee p_0 \geq 0)$ **then return** $6 \cdot 13^5 + t_0 \cdot 13^4 + \max(t_1, p_0) \cdot 13^3$ ▷ *Full House*
$st \leftarrow \texttt{STR8}[all]$
if $st \geq 0$ **then return** $4 \cdot 13^5 + st \cdot 13^4$ ▷ *Straight*
if $t_0 \geq 0$ **then return** $3 \cdot 13^5 + t_0 \cdot 13^4 + k_0 \cdot 13^3 + k_1 \cdot 13^2$ (top 2 kickers $k_0 > k_1$ with $cnt[k] = 1$)
if $p_1 \geq 0$ **then return** $2 \cdot 13^5 + p_0 \cdot 13^4 + p_1 \cdot 13^3 + (\max_{r \notin \{p_0,p_1\}, cnt[r]>0} r) \cdot 13^2$ ▷ *Two Pair*
if $p_0 \geq 0$ **then return** $1 \cdot 13^5 + p_0 \cdot 13^4 + k_0 \cdot 13^3 + k_1 \cdot 13^2 + k_2 \cdot 13$ (top 3 kickers $k_0 > k_1 > k_2$)
return $\texttt{TOP5T}[all]$ ▷ *High Card* ($\kappa = 0$)

Proposition 4.3 (Pigeonhole Suit Uniqueness and Early Flush Return in RANK7FAST)**.** *In any 7-card hand A ($|A| \leq 7$), at most one suit $u \in \{0,1,2,3\}$ can satisfy $\texttt{POPC}[s_u] \geq 5$, and whenever such a suit exists, no Four-of-a-Kind ($\kappa = 7$) or Full House ($\kappa = 6$) can simultaneously exist in A.*

Proof. First, if at least 5 cards were contained in each of two distinct suits, at least $5+5 = 10 > 7$ cards would be required, contradicting $|A| \leq 7$. Second, suppose suit u has $\texttt{POPC}[s_u] \geq 5$ cards. Since all cards in s_u share suit u, their ranks must be 5 distinct ranks r_1, r_2, r_3, r_4, r_5. At most $7-5 = 2$ cards remain in $A \setminus s_u$. Even if ranks in s_u are paired by both remaining cards, the maximum multiplicity profile across all 13 ranks is either $(3,1,1,1,1)$ (Three of a Kind) or $(2,2,1,1,1)$ (Two Pair); neither $(4,1,\ldots)$ nor $(3,2,\ldots)$ can be formed with only 2 extra cards. Hence checking $f_m \neq 0$ before checking for Four-of-a-Kind or Full House is mathematically exact. □

To convert a packed integer score $v = V(\kappa, k)$ back into a human-readable category and rank description, our inverse process (DECODE, Algorithm 7) extracts the base-13 digits from least significant to most significant.

Algorithm 7: DECODE — unpack a base-13 score into its category and tie-breaker ranks

Input: packed score $v \in [0,\ 3,341,636]$
Output: category $\kappa \in \{0,\ldots,8\}$ and tie-breaker array $k = (k_0, k_1, k_2, k_3, k_4)$

$x \leftarrow v$
for $i \leftarrow 4$ **downto** 0 **do**
  $k_i \leftarrow x \bmod 13$; $x \leftarrow \lfloor x/13 \rfloor$
$\kappa \leftarrow x$
return (κ, k)

4.5 Processes in Associated Standard Hand Evaluators

While RANK7FAST achieves **1.77 million** 7-card evaluations per second using only **48 KB** of lookup tables ($3 \times 8,192$ 8/32-bit integers, fitting inside L1 CPU cache), larger memory footprints are traded for even fewer instructions per hand in standalone C++ and TypeScript evaluators. Three standard evaluator architectures are compared alongside our evaluator in Table 3:

1. **The Prime-Product Perfect-Hash Process (Cactus Kev / Paul Senzee):** The first 13 prime numbers are assigned to the 13 ranks:

$$(2, 3, 4, 5, 6, 7, 8, 9, \mathrm{T}, \mathrm{J}, \mathrm{Q}, \mathrm{K}, \mathrm{A}) \mapsto (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41).$$

   Each card is encoded as a 32-bit word packing its prime p_r, rank nibble r, suit bit 2^σ, and rank bit 2^{16+r}. For any 5-card hand, if all 5 cards share a suit bit ($(c_1 \,\&\, c_2 \,\&\, c_3 \,\&\, c_4 \,\&\, c_5 \,\&\, \texttt{0xF000}) \neq 0$), a single 8,192-entry lookup on the bitwise OR of the rank bits returns the Flush or Straight-Flush rank. Otherwise, if all 5 rank bits are distinct ($\texttt{unique5}[\mathit{rankOr}] \neq 0$), the Straight or High-Card rank is returned immediately. For paired hands, it is guaranteed by the Fundamental Theorem of Arithmetic that the product of the five primes $P = \prod_{i=1}^{5} p_{r_i}$ uniquely identifies the rank multiset among the 4,888 possible paired 5-card rank multisets, which is mapped in $O(1)$ by Paul Senzee's perfect hash without binary search.

2. **The Quinary Combinadic Dynamic-Programming Process (`PHEvaluator` / `@pokertools`):** In HenryRLee's `PokerHandEvaluator` (`PHEvaluator`) and `@pokertools/evaluator`, 7-card hands are evaluated directly without iterating over 21 subsets. A 7-card hand's rank histogram is represented as a 13-tuple $q = (q_0, q_1, \ldots, q_{12}) \in \{0, 1, 2, 3, 4\}^{13}$ satisfying $\sum_{r=0}^{12} q_r = 7$ (a 13-digit base-5, or **quinary**, number). By the generating function coefficient $[x^7](1 + x + x^2 + x^3 + x^4)^{13} = [x^7](1 - x^5)^{13}(1 - x)^{-13}$, the exact number of valid 13-tuples is:

$$N_{\text{quinary}}(13, 7) = \binom{19}{7} - 13\binom{14}{2} = 50,388 - 1,183 = \mathbf{49,205}. \tag{15}$$

   Using a precomputed dynamic-programming table $\mathrm{DP}[\ell][r][k]$, the quinary vector q is mapped bijectively into $\{0, \ldots, 49,204\}$ via:

$$h(q) = \sum_{r=0}^{12} \mathrm{DP}[q_r][r] \left[7 - \sum_{j=0}^{r-1} q_j \right]. \tag{16}$$

   Looking up $\texttt{NO_FLUSH_7}[h(q)]$ returns the exact equivalence class in 1..7,462 in $O(1)$ using only $\approx$ 2.5–5 **MB** of tables.

3. **The 7-Step Directed Acyclic State-Graph Process (`TwoPlusTwo` / `HandRanks.dat`):** The entire 7-card evaluation process is compiled into a deterministic finite automaton stored as a flat array `HR` of 32,487,834 32-bit integers (**128 MB**). Given 7 cards $c_1, \ldots, c_7 \in \{1, \ldots, 52\}$, evaluation is reduced to seven chained array dereferences with zero branches:

$$p \leftarrow \texttt{HR}\Big[\texttt{HR}\big[\ldots \texttt{HR}[\texttt{HR}[53 + c_1] + c_2] \ldots\big] + c_7\Big]. \tag{17}$$

5 Process III: Equity Estimation, Multi-Way Bunching, and Runout Enumeration

5.1 Starting-Hand Isomorphism (1,326 to 169 Classes), Card Removal, and Outs

Before community cards are dealt, $\binom{52}{2} = \frac{52 \times 51}{2} = 1,326$ distinct two-card starting hands exist. Because suits have identical strategic value preflop, any two hands that differ only by a permutation

Evaluation Process	Core Mechanism	Table Size	Throughput	Architectural Trade-Off
21-Subset EVAL5	$\binom{7}{5} = 21$ subset loops	0 KB	0.024M / s	Zero tables; returns exact 5 winning cards for showdown display.
Bitmask RANK7FAST	Suit/rank masks + `TOP5T`	48 KB	**1.77**M / s	**75**× speedup over EVAL5; fits entirely inside L1 CPU cache.
Prime-Product Hash	$\prod_{i=1}^{5} p_{r_i}$ + perfect hash	≈ 150 KB	4M–8M / s	Fast for 5-card hands; still checks $\binom{7}{5} = 21$ subsets for 7-card hands.
Quinary Combinadic	Base-5 DP hash (49,205)	2.5–5 MB	10M–115M / s	Evaluates 7 cards directly via Eq. (16) with moderate memory.
State-Graph DAG	7 chained array lookups	128 MB	60M–200M / s	Branchless 7-step pointer chase; requires 128 MB binary asset.

Table 3: Comparison of 7-card hand evaluation processes across memory footprint and throughput.

of suits are **strategically isomorphic**. When the 1,326 combinations are grouped by rank pair and suitedness, they collapse into 169 **canonical starting-hand classes**:

$$\underbrace{13 \text{ Pairs}}_{13\times 6=78} + \underbrace{\binom{13}{2} = 78 \text{ Suited}}_{78\times 4=312} + \underbrace{\binom{13}{2} = 78 \text{ Offsuit}}_{78\times 12=936} = 169 \text{ classes } (1{,}326 \text{ combos}). \tag{18}$$

Thus, at random deal, a pocket pair is dealt with probability $78/1{,}326 = 1/17 \approx 5.88\%$, a specific pocket pair such as A A is dealt with probability $6/1{,}326 = 1/221 \approx 0.452\%$, a specific suited hand such as A Ks with probability $4/1{,}326 \approx 0.302\%$, and a specific offsuit hand such as A Ko with probability $12/1{,}326 \approx 0.905\%$.

On the flop (3 board cards) or turn (4 board cards), a hand that is currently behind may have O unseen **outs** remaining in the stub that will improve it to a winning hand. With $U = 52 - 2 - |B|$ unseen cards ($U = 47$ on the flop, $U = 46$ on the turn), the exact hypergeometric probability of hitting at least one out is given by:

$$\mathbb{P}(\text{hit turn}) = \frac{O}{47}, \qquad \mathbb{P}(\text{hit river}) = \frac{O}{46}, \qquad \mathbb{P}(\text{hit by river from flop}) = 1 - \frac{(47-O)(46-O)}{47\times 46}. \tag{19}$$

Plain-English Intuition: The Rule of 4 and 2 (and Why We Simulate Full Equity). Because $\frac{1}{47} \approx 2.13\%$ and $\frac{1}{46} \approx 2.17\%$, outs O are often multiplied by 4 on the flop (to estimate turn-plus-river hit probability) and by 2 on the turn (to estimate single-card river hit probability). For instance, a 9-out flush draw hits by the river with exact probability $1 - \frac{38\times 37}{47\times 46} = 34.97\% \approx 9 \times 4\% = 36\%$. However, we never rely on raw out counts inside our engine because some outs simultaneously give an opponent a better hand (**dirty outs**, such as completing a flush when the board pairs to give an opponent a full house), and other runouts win via runner-runner cards or split pots. Instead, true **pot-share equity** is simulated across full runouts.

5.2 The Monte Carlo Rollout Process (EQUITY) and Bessel-Corrected Standard Error

Suppose we hold $H = \{h_1, h_2\}$ on a partial board B ($|B| \in \{0, 3, 4, 5\}$) against $m \geq 1$ active opponents. Let $K = 52 - |H \cup B|$ denote the size of the unseen stub $D = \{0, \ldots, 51\} \setminus (H \cup B)$, and let $b_{\text{need}} = 5 - |B|$ denote the number of community cards still to be dealt. Each simulated runout requires drawing $k = b_{\text{need}} + 2m$ cards uniformly without replacement from D.

Rather than copying or re-initializing the K-card array D on every trial, our Monte Carlo rollout process (Algorithm 8) allocates D **once** before the loop and performs a **partial Fisher–Yates shuffle of only the first k slots** in-place across all N trials. Because each trial yields a fractional pot share $w_t \in \{0, 1, 1/2, 1/3, \ldots\}$, we compute the unbiased **Bessel-corrected sample variance** with denominator $N - 1$:

$$\hat{q} = \frac{W_1}{N}, \qquad s^2 = \max\left(0,\ \frac{W_2 - N\hat{q}^2}{N-1}\right), \qquad \mathrm{SE}(\hat{q}) = \sqrt{\frac{s^2}{N}}. \tag{20}$$

Algorithm 8: EQUITY — Monte Carlo pot-share estimator with in-place partial Fisher–Yates sampling and Bessel $(N-1)$ SE

Input: focal hole cards $H = (h_1, h_2)$; current board B ($|B| \leq 5$); opponent count $m \geq 1$; trial budget N

Output: estimated pot-share equity $\hat{q} \in [0, 1]$, sample variance s^2, and standard error $\mathrm{SE} = \sqrt{s^2/N}$

Build stub array $D[0 \ldots K-1] \leftarrow \{0, \ldots, 51\} \setminus (H \cup B)$, where $K = 50 - |B|$
$b_{\text{need}} \leftarrow 5 - |B|$; $k \leftarrow b_{\text{need}} + 2m$ ▷ *cards needed per trial*
Pre-fill $simH[0, 1] \leftarrow h_1, h_2$ and $simH[2 \ldots 1 + |B|] \leftarrow B$; similarly pre-fill $oppH[2 \ldots 1 + |B|] \leftarrow B$
$W_1 \leftarrow 0$; $W_2 \leftarrow 0$ ▷ *running sums $\sum w_t$ and $\sum w_t^2$*
for $t \leftarrow 1$ **to** N **do**
  for $i \leftarrow 0$ **to** $k - 1$ **do** ▷ *partial Fisher–Yates over first k slots*
    $j \leftarrow i + \lfloor U \cdot (K - i) \rfloor$, where $U \sim \mathrm{Uniform}[0, 1)$
    Swap $D[i] \leftrightarrow D[j]$
  Copy $D[0 \ldots b_{\text{need}} - 1]$ into the remaining board slots of $simH$ and $oppH$
  $v_{\text{hero}} \leftarrow \text{RANK7FAST}(simH)$; $bestOpp \leftarrow -1$; $ties \leftarrow 0$
  for $p \leftarrow 0$ **to** $m - 1$ **do**
    $oppH[0] \leftarrow D[b_{\text{need}} + 2p]$; $oppH[1] \leftarrow D[b_{\text{need}} + 2p + 1]$
    $v_p \leftarrow \text{RANK7FAST}(oppH)$
    if $v_p > bestOpp$ **then** $bestOpp \leftarrow v_p$; $ties \leftarrow 1$ **else if** $v_p = bestOpp$ **then** $ties \leftarrow ties + 1$
  $w_t \leftarrow 1$ if $v_{\text{hero}} > bestOpp$, else $\frac{1}{ties+1}$ if $v_{\text{hero}} = bestOpp$, else 0
  $W_1 \leftarrow W_1 + w_t$; $W_2 \leftarrow W_2 + w_t^2$
$\hat{q} \leftarrow W_1/N$; $s^2 \leftarrow \max\big(0, (W_2 - N\hat{q}^2)/(N-1)\big)$; $\mathrm{SE} \leftarrow \sqrt{s^2/N}$
return $(\hat{q}, s^2, \mathrm{SE})$

Proposition 5.1 (Correctness of Non-Resetting Partial Fisher–Yates Across Trials)**.** *In Algorithm 8, mutating D in-place across trials $t = 1, \ldots, N$ without resetting D to sorted order between trials still produces an independent, strictly uniform k-tuple of distinct cards from D on every trial.*

Proof. At the start of any trial t, because elements within D were only ever swapped by prior trials, the array $D[0 \ldots K-1]$ remains a bijection (permutation) of the fixed set of K unseen cards. When k steps of Fisher–Yates are executed on any initial bijection of K distinct elements, each of the $K(K-1)\cdots(K-k+1)$ ordered k-tuples is mapped into $D[0 \ldots k-1]$ with probability $\prod_{i=0}^{k-1} \frac{1}{K-i}$, independent of the initial permutation of D. □

Proposition 5.2 (Variance Reduction over Fractional Split-Pot Outcomes)**.** *Let $w_t \in [0,1]$ be the fractional pot share won on trial t. Whenever split pots ($0 < w_t < 1$) occur, $w_t^2 < w_t$, so the true pot-share variance is strictly smaller than the binary Bernoulli approximation $\hat{q}(1-\hat{q})$. In particular, on a board-locked chop where every trial ties ($w_t = 1/2$), Eq. (20) yields the exact $s^2 = 0$ and* SE $= 0$.

Proof. For every $w_t \in (0,1)$, $w_t^2 < w_t$, so $\sum_{t=1}^{N} w_t^2 \le \sum_{t=1}^{N} w_t = N\hat{q}$, with strict inequality whenever at least one trial splits the pot. On a locked chop board where $w_t = 1/2$ for all t, $W_1 = N/2$ and $W_2 = N/4$, giving $W_2 - N\hat{q}^2 = N/4 - N(1/4) = 0 \implies s^2 = 0$. □

5.3 Range Rejection Sampling, Multi-Way Bunching, and Board-Lock Protection

Uniform Monte Carlo (EQUITY) assumes that an opponent is just as likely to hold 7♠ 2♢ as A♠ A♡. Once an opponent voluntarily calls or raises preflop, their range is skewed toward stronger starting hands. Moreover, on a 4–8 player table, when several early-position players fold before the action reaches the callers, the mucked cards are heavily biased toward low ranks—a phenomenon known as **card-removal bunching**.

In our upgraded equity architecture, three specialized algorithms are combined:

1. **Single-Opponent Range Rejection Sampling (EQUITYVSRANGE, Algorithm 9):** Uses our precomputed 52×52 matrix `CHEN_PAIR`$[c_1, c_2]$ to reject sampled opponent hole cards below an action-conditioned floor c_{floor}.
2. **Multi-Way Range Conditioning with Preflop Bunching (EQUITYMULTIWAYRANGED, Algorithm 10):** First draws $2 \times n_{\text{folded}}$ cards biased toward Chen ≤ 5.0 to model the folded preflop muck on 4–8 player tables, then rejection-samples each active opponent $o \in \{0, \dots, m-1\}$ against their individual floor $c_{\text{floor}}(o)$, and tracks $(\mathbb{P}(\text{win}), \mathbb{P}(\text{tie}), \mathbb{P}(\text{loss}))$ separately.
3. **Board-Lock Split-Pot Protection (ISBOARDLOCKNUTCHOP, Algorithm 11):** Checks on a 5-card river board whether the board itself is the nuts ($\mathbb{P}(\text{loss}) = 0$) so that a guaranteed chop is never folded to an overbet shove.

Algorithm 9: EQUITYVSRANGE — rejection-sampled Monte Carlo equity against an action-conditioned range floor

---

Input: hero hole cards H; board B; opponent Chen floor c_{floor}; trial budget N
Output: conditional equity $\hat{q}(H \mid \text{Chen}(H_{\text{opp}}) \ge c_{\text{floor}})$, or **nil** if sample size < 30

Build stub $D[0 \dots K-1] \leftarrow \{0, \dots, 51\} \setminus (H \cup B)$; $b_{\text{need}} \leftarrow 5 - |B|$
$W_1 \leftarrow 0$; $n_{\text{acc}} \leftarrow 0$; $n_{\text{att}} \leftarrow 0$; $N_{\max} \leftarrow 18N$
while $n_{\text{acc}} < N \wedge n_{\text{att}} < N_{\max}$ **do**
  $n_{\text{att}} \leftarrow n_{\text{att}} + 1$
  Draw 2 opponent cards $D[0], D[1]$ via 2 Fisher–Yates swaps on $D[0 \dots K-1]$
  if `CHEN_PAIR`$[D[0], D[1]] < c_{\text{floor}}$ **then continue** ▷ *reject hands below range floor*
  Draw b_{need} board cards $D[2 \dots 1 + b_{\text{need}}]$ via Fisher–Yates swaps on $D[2 \dots K-1]$
  $v_{\text{hero}} \leftarrow \text{RANK7FAST}(H \cup B \cup D[2 \dots 1 + b_{\text{need}}])$
  $v_{\text{opp}} \leftarrow \text{RANK7FAST}(\{D[0], D[1]\} \cup B \cup D[2 \dots 1 + b_{\text{need}}])$
  $W_1 \leftarrow W_1 + (1 \text{ if } v_{\text{hero}} > v_{\text{opp}},\ 0.5 \text{ if } v_{\text{hero}} = v_{\text{opp}},\ \text{else } 0)$; $n_{\text{acc}} \leftarrow n_{\text{acc}} + 1$
return W_1 / n_{acc} **if** $n_{\text{acc}} \ge 30$ **else nil**

Algorithm 10: EQUITYMULTIWAYRANGED — multi-way range-conditioned rollout with preflop card-removal bunching

Input: hero hole cards H; board B; opponent floors $(c_{\text{floor}}(0), \ldots, c_{\text{floor}}(m-1))$; folded count n_{fold}; budget N

Output: pot-share equity $\hat{q}$, SE, and decomposed $(\mathbb{P}(\text{win}), \mathbb{P}(\text{tie}), \mathbb{P}(\text{loss}))$

Build stub $D[0 \ldots K-1] \leftarrow \{0, \ldots, 51\} \setminus (H \cup B)$; $b_{\text{need}} \leftarrow 5 - |B|$; $n_b \leftarrow \min(4, n_{\text{fold}})$
while $n_{\text{acc}} < N \wedge n_{\text{att}} < N_{\max}$ **do**
  $ptr \leftarrow 0$
  for $f \leftarrow 1$ **to** n_b **do** ▷ *sample bunched preflop folds biased toward* Chen ≤ 5.0
    Sample pair $D[ptr], D[ptr+1]$ up to 3 tries until CHEN_PAIR$[D[ptr], D[ptr+1]] \leq 5.0$; $ptr \mathrel{+}= 2$
  for $o \leftarrow 0$ **to** $m-1$ **do** ▷ *sample each active opponent's range*
    Sample pair $D[ptr], D[ptr+1]$ until CHEN_PAIR$[D[ptr], D[ptr+1]] \geq c_{\text{floor}}(o)$; $ptr \mathrel{+}= 2$
  Draw b_{need} board cards $D[ptr \ldots ptr + b_{\text{need}} - 1]$ and score hero vs. all m opponents via RANK7FAST
  Accumulate $w_t \in \{0, 1/(1+ties), 1\}$ into W_1, W_2 and increment win/tie/loss counters
return $(\hat{q}$, SE, $\mathbb{P}(\text{win})$, $\mathbb{P}(\text{tie})$, $\mathbb{P}(\text{loss}))$

Algorithm 11: ISBOARDLOCKNUTCHOP — detect whether the 5-card river board itself is the unbeatable nuts

Input: hero hole cards $H = (h_1, h_2)$; five-card river board B_5

Output: boolean **true** if $\mathbb{P}(\text{loss}) = 0$ (the board plays and no unseen pair beats B_5), else **false**

$v_{\text{board}} \leftarrow \text{EVAL5}(B_5)$
if RANK7FAST$(H \cup B_5) \neq v_{\text{board}}$ **then return false**
$D \leftarrow \{0, \ldots, 51\} \setminus (H \cup B_5)$
for $i \leftarrow 0$ **to** $|D| - 2$ **do for** $j \leftarrow i+1$ **to** $|D| - 1$ **do**
  if RANK7FAST$(\{D_i, D_j\} \cup B_5) > v_{\text{board}}$ **then return false**
return true

5.4 The Exact Turn/River Enumeration and Next-Card Vulnerability Process

When the board reaches the river ($|B| = 5$) or turn ($|B| = 4$) in a heads-up pot ($m = 1$), the remaining sample space is small enough that random simulation is replaced in our engine by **exact deterministic enumeration** (SE $= 0$, Algorithm 12):

- **River ($|B| = 5$, 45 unseen cards):** All $\binom{45}{2} = \frac{45 \times 44}{2} =$ **990** possible opponent two-card holdings are enumerated in < 0.6 ms.
- **Turn ($|B| = 4$, 46 unseen cards):** For each of the 46 possible river cards $c \in D$, all $\binom{45}{2} = 990$ opponent holdings are enumerated, requiring $46 \times 990 =$ **45, 540** evaluations (≈ 24 ms in JS). Averaging the 46 exact conditional equities $q(c)$ yields exact turn equity $q_{\text{base}} = \frac{1}{46} \sum_{c \in D} q(c)$ with zero sampling error (SE $= 0$) and classifies every unseen river card c as a **helpful card** ($q(c) - q_{\text{base}} > 0.03$, i.e., an effective out), a **scare card** ($q(c) - q_{\text{base}} < -0.03$), or neutral.
- **Flop ($|B| = 3$, 47 unseen cards) or Multi-Way Turn ($m \geq 2$):** Monte Carlo rollouts are executed ($N_{\text{base}} = 500$ trials for baseline equity and $N_{\text{per}} = 200$ trials per candidate card $c \in D$) and the classification threshold is widened dynamically to the 2σ noise floor of the difference between two independent Monte Carlo estimators:

$$\tau = \max\left(0.03,\ 2\sqrt{\text{SE}_{\text{base}}^2 + \text{SE}_c^2}\right) \approx 0.036\text{–}0.070, \tag{21}$$

so that random sampling jitter never mislabels a neutral card as an out or scare card.

Algorithm 12: NEXTCARDSCAN — exact turn/river enumeration and dynamic 2σ next-card vulnerability scanner

Input: hole cards $H = (h_1, h_2)$; board B ($|B| \in \{3, 4\}$); active opponents $m \geq 1$
Output: baseline equity q_{base}, helpful outs count n_+, scare card count n_-, and per-rank deltas

$D \leftarrow \{0, \ldots, 51\} \setminus (H \cup B)$
if $|B| = 4 \wedge m = 1$ **then** ▷ *exact deterministic enumeration over* $46 \times \binom{45}{2} = 45{,}540$ *deals*
  foreach candidate river card $c \in D$ **do**
    $B' \leftarrow B \cup \{c\}$; $R \leftarrow D \setminus \{c\}$; $v_{\text{hero}} \leftarrow \text{RANK7FAST}(H \cup B')$; $W \leftarrow 0$
    for $i \leftarrow 0$ **to** 43 **do for** $j \leftarrow i + 1$ **to** 44 **do**
      $v_{\text{opp}} \leftarrow \text{RANK7FAST}(\{R_i, R_j\} \cup B')$
      $W \leftarrow W + (1 \text{ if } v_{\text{hero}} > v_{\text{opp}},\ 0.5 \text{ if } v_{\text{hero}} = v_{\text{opp}},\ \text{else } 0)$
    $q(c) \leftarrow W/990$; $\text{SE}_c \leftarrow 0$
  $q_{\text{base}} \leftarrow \frac{1}{46} \sum_{c \in D} q(c)$; $\text{SE}_{\text{base}} \leftarrow 0$
else ▷ *Monte Carlo rollout with dynamic* 2σ *noise filter*
  $(q_{\text{base}}, _, \text{SE}_{\text{base}}) \leftarrow \text{EQUITY}(H, B, m, N_{\text{base}})$
  foreach $c \in D$ **do** $(q(c), _, \text{SE}_c) \leftarrow \text{EQUITY}(H, B \cup \{c\}, m, N_{\text{per}})$
$n_+ \leftarrow 0$; $n_- \leftarrow 0$
foreach $c \in D$ **do**
  $\tau_c \leftarrow \max\left(0.03,\ 2\sqrt{\text{SE}_{\text{base}}^2 + \text{SE}_c^2}\right)$; $\Delta_c \leftarrow q(c) - q_{\text{base}}$
  if $\Delta_c > \tau_c$ **then** $n_+ \leftarrow n_+ + 1$ **else if** $\Delta_c < -\tau_c$ **then** $n_- \leftarrow n_- + 1$
return $(q_{\text{base}}, n_+, n_-)$

6 Process IV: The Betting State Machine and Turn-Order Process

6.1 Player Chip Accounting and the TDA Rule 19 Dead-Button Blind Process (STARTHAND)

To prevent chip-accounting discrepancies across multi-street betting and side-pot settlement, three distinct integer counters are maintained on every player record p:

- $p.\texttt{stack} \geq 0$: Chips currently sitting in front of player p available to be wagered.
- $p.\texttt{bet} \geq 0$: Chips wagered by player p **on the current street only** (reset to 0 at the start of the flop, turn, and river).
- $p.\texttt{committed} \geq 0$: Cumulative chips wagered by player p **across the entire hand** (never reset until the hand ends; only reduced if an uncalled overbet is refunded).

Whenever δ chips are placed into the pot by player p, all three counters are updated atomically via the conservation rule:

$$p.\texttt{stack} \mathrel{-}= \delta, \qquad p.\texttt{bet} \mathrel{+}= \delta, \qquad p.\texttt{committed} \mathrel{+}= \delta, \qquad \texttt{pot} \mathrel{+}= \delta. \tag{22}$$

When a player busts ($p.\texttt{stack} = 0$ at the end of a hand), naively stepping the dealer button or small blind backward to the previous surviving seat can cause a player who just posted the big blind to post the small blind on the very next hand, or allow a surviving player to skip a blind. Under official **Poker Tournament Directors Association (TDA) Rule 19 (Dead-Button /**

Forward-Moving Blinds), we anchor the new **small blind seat directly to the previous hand's big blind seat** (`sbSeat` $\leftarrow$ `prevBbSeat`, whether that chair is still occupied or now an empty "dead small blind" seat), advance the new **big blind to the next funded seat clockwise** (`bbSeat` $\leftarrow$ NEXTFUNDEDSEATCLOCKWISE(`sbSeat`)), and place the button one seat counter-clockwise from the small blind (Algorithm 13). Furthermore, if the big blind player has a short stack ($p_{\texttt{bb}}.\texttt{stack} < \texttt{bb}$), `currentBet` is still initialized to the full `bb` so the next player cannot limp in for less than a full big blind.

Algorithm 13: STARTHAND — TDA Rule 19 dead-button blind advancement, short-blind bring-in, and preflop initialisation

Input: previous big blind seat `prevBbSeat`; funded seat list $E = (e_0, \ldots, e_{k-1})$ on a 4–8 player table ($N_p \in \{4..8\}$)

Output: updated (`btnSeat`, `sbSeat`, `bbSeat`), posted blinds, and first actor `actingIdx`

if `prevBbSeat` $= -1$ **then** `btnSeat` $\leftarrow e_0$; `sbSeat` $\leftarrow (e_0$ if $k = 2$ else $e_1)$; `bbSeat` $\leftarrow (e_1$ if $k = 2$ else $e_2)$

else if $k = 2$ **then** ▷ *heads-up: alternate blinds, BTN posts SB*

  `bbSeat` $\leftarrow$ NEXTFUNDEDSEAT(`prevBbSeat`); `sbSeat` $\leftarrow$ NEXTFUNDEDSEAT(`bbSeat`); `btnSeat` $\leftarrow$ `sbSeat`

else ▷ *TDA Rule 19: SB takes previous BB seat; BB advances to next live seat*

  `sbSeat` $\leftarrow$ `prevBbSeat`; `bbSeat` $\leftarrow$ NEXTFUNDEDSEAT(`sbSeat`); `btnSeat` $\leftarrow (\texttt{sbSeat} - 1 + N_p) \bmod N_p$

if $p_{\texttt{sb}}.\texttt{dealtIn}$ **then** POSTBLIND($p_{\texttt{sb}}$, `sb`) ▷ *skipped if dead small blind*

POSTBLIND($p_{\texttt{bb}}$, `bb`)

`currentBet` $\leftarrow \max(\texttt{bb}, p_{\texttt{bb}}.\texttt{bet})$; `lastRaise` $\leftarrow$ `bb` ▷ *full BB bring-in even if BB is short*

`actingIdx` $\leftarrow$ FINDNEXTACTOR(`bbSeat`) ▷ *UTG acts first preflop (SB/BTN in heads-up)*

6.2 The Turn-Order and Round-Completion Process (FINDNEXTACTOR)

After any player acts (or at the start of a street), our turn-order process (FINDNEXTACTOR, Algorithm 14) scans seats clockwise starting after seat *from*. A dealt-in, non-folded, non-all-in player p requires an action if and only if the **Pending Action Disjunction** is satisfied:

$$\text{NeedsAct}(p) \iff p.\texttt{dealtIn} \wedge \neg p.\texttt{folded} \wedge \neg p.\texttt{allIn} \wedge (\neg p.\texttt{hasActed} \vee p.\texttt{bet} < \texttt{currentBet}). \tag{23}$$

Algorithm 14: FINDNEXTACTOR — determine next seat to act or signal end of betting street

Input: seat index *from*; player array $P[0 \ldots N_p - 1]$; street bet `currentBet`

Output: next seat index $idx \in \{0, \ldots, N_p - 1\}$, or -1 if the betting street is complete

$n_{\text{live}} \leftarrow |\{p \in P : p.\texttt{dealtIn} \wedge \neg p.\texttt{folded}\}|$; $n_{\text{canAct}} \leftarrow |\{p \in P : p.\texttt{dealtIn} \wedge \neg p.\texttt{folded} \wedge \neg p.\texttt{allIn}\}|$

if $n_{\text{live}} \leq 1 \vee n_{\text{canAct}} = 0$ **then return** -1 ▷ *hand over or all remaining players all-in*

if $n_{\text{canAct}} = 1$ and the sole active player $p^\star$ satisfies $p^\star.\texttt{bet} \geq \texttt{currentBet}$ **then return** -1

for $k \leftarrow 1$ **to** N_p **do**

  $idx \leftarrow (from + k) \bmod N_p$; $p \leftarrow P[idx]$

  if $\neg p.\texttt{dealtIn} \vee p.\texttt{folded} \vee p.\texttt{allIn}$ **then continue**

  if $\neg p.\texttt{hasActed} \vee p.\texttt{bet} < \texttt{currentBet}$ **then return** idx

return -1 ▷ *every active player has acted and matched* `currentBet`

6.3 The Action Application and Cumulative Short All-In Reopening Process

A classic edge case in No-Limit Hold'em concerns **short all-in raises**. Suppose the big blind is 20, Player A bets 100 ($\texttt{lastRaise} = 100$), and Player B moves all-in for 140 total (an increase of only $40 < 100$). When the action returns to Player A, 40 more chips are owed ($p_A.\texttt{bet} = 100 < 140$), so Player A is given a turn by FINDNEXTACTOR. However, because Player B's all-in increase (40) was less than the full raise increment (100), **Player A is only permitted to call the** 40 **or fold; re-raising is prohibited!** What if Player C also goes all-in for 210 total after Player B? Individually, neither $140 - 100 = 40$ nor $210 - 140 = 70$ is a full 100 raise, but **cumulatively** the bet has risen from 100 to 210 ($210 - 100 = 110 \geq 100$), by which raising **is** reopened for Player A under Poker TDA Rules.

To enforce both individual and cumulative reopening cleanly, $p.\texttt{actedAtBet}$ (initialized to -1 at the start of each street and set to $\texttt{currentBet}$ after each action by p) is recorded on every player. When a turn returns to p, CANREOPEN(p) (Algorithm 15) simply checks whether $p.\texttt{actedAtBet} < 0$ or $(\texttt{currentBet} - p.\texttt{actedAtBet}) \geq \texttt{lastRaise}$.

Algorithm 15: APPLYACTIONANDREOPEN — state transition for player actions and cumulative short all-in reopening check

Input: player p; action (a, τ); state $(\texttt{currentBet}, \texttt{lastRaise}, \texttt{raises})$
Output: updated betting state and next actor

Function CANREOPEN(p):
  if $p.\texttt{stack} \leq \max(0, \texttt{currentBet} - p.\texttt{bet})$ **then return false** ▷ *no chips beyond a call*
  if $p.\texttt{actedAtBet} < 0$ **then return true** ▷ *has not yet acted on this street*
  return $(\texttt{currentBet} - p.\texttt{actedAtBet}) \geq \texttt{lastRaise}$ ▷ *cumulative short all-ins ≥ full raise*
Procedure APPLYACTION(p, a, τ):
  if $a =$ FOLD **then** $p.\texttt{folded} \leftarrow$ **true**; $p.\texttt{hasActed} \leftarrow$ **true**
  else if $a =$ CHECK **then** $p.\texttt{hasActed} \leftarrow$ **true**
  else if $a =$ CALL **then**
    $\delta \leftarrow \min(p.\texttt{stack}, \max(0, \texttt{currentBet} - p.\texttt{bet}))$
    Transfer δ chips via Eq. (22); **if** $p.\texttt{stack} = 0$ **then** $p.\texttt{allIn} \leftarrow$ **true**
    $p.\texttt{hasActed} \leftarrow$ **true**
  else if $a =$ RAISE **then**
    $\tau_{\max} \leftarrow p.\texttt{bet} + p.\texttt{stack}$; $\tau_{\min} \leftarrow \min(\tau_{\max}, \max(\texttt{currentBet} + \texttt{lastRaise}, \texttt{bb}))$
    $\tau^\star \leftarrow \min(\tau_{\max}, \max(\tau_{\min}, \tau))$; $\delta \leftarrow \tau^\star - p.\texttt{bet}$; $\Delta \leftarrow \tau^\star - \texttt{currentBet}$
    Transfer δ chips via Eq. (22); **if** $p.\texttt{stack} = 0$ **then** $p.\texttt{allIn} \leftarrow$ **true**
    if $\tau^\star > \texttt{currentBet}$ **then**
      if $\Delta \geq \texttt{lastRaise}$ **then** $\texttt{lastRaise} \leftarrow \Delta$ ▷ *full raise updates* $\texttt{lastRaise}$
      foreach $q \neq p$ **with** $q.\texttt{dealtIn} \wedge \neg q.\texttt{folded} \wedge \neg q.\texttt{allIn}$ **do** $q.\texttt{hasActed} \leftarrow$ **false**
      $\texttt{currentBet} \leftarrow \tau^\star$; $\texttt{raises} \leftarrow \texttt{raises} + 1$; $\texttt{aggressor} \leftarrow p$
    $p.\texttt{hasActed} \leftarrow$ **true**
  $p.\texttt{actedAtBet} \leftarrow \texttt{currentBet}$ ▷ *record bet level faced upon completing action*

7 Process V: Uncalled Bet Refund, Side-Pot Slicing, and Showdown Settlement

7.1 The Uncalled Bet Refund Process (REFUNDUNCALLED)

At the close of every betting round and before any pots are sliced at showdown, if the player $p_{\max}$ with the strictly highest commitment c_1 has committed more than the second-highest commitment

c_2, no opponent matched the excess $\Delta_{\text{uncalled}} = c_1 - c_2$. Under universal poker rules, those Δ_{uncalled} chips are **not part of any pot**; they are immediately returned to $p_{\max}$ (Algorithm 16):

$$p_{\max}.\texttt{committed} \mathrel{-}= \Delta_{\text{uncalled}}, \qquad p_{\max}.\texttt{stack} \mathrel{+}= \Delta_{\text{uncalled}}, \qquad \texttt{pot} \mathrel{-}= \Delta_{\text{uncalled}}. \tag{24}$$

Algorithm 16: REFUNDUNCALLED — return unmatched excess wager before pot decomposition

Input: player array P

Output: excess chips Δ_{uncalled} returned to the sole highest committer

Sort committers $C \leftarrow \{p \in P : p.\texttt{dealtIn} \wedge p.\texttt{committed} > 0\}$ in descending order of $p.\texttt{committed}$

if $|C| = 0$ **then return**

$c_1 \leftarrow C_0.\texttt{committed}$; $\quad c_2 \leftarrow (C_1.\texttt{committed}$ if $|C| \geq 2$ else $0)$

if $c_1 > c_2$ **then**

  $\Delta \leftarrow c_1 - c_2$

  $C_0.\texttt{committed} \leftarrow c_2$; $\quad C_0.\texttt{stack} \leftarrow C_0.\texttt{stack} + \Delta$; $\quad \texttt{pot} \leftarrow \texttt{pot} - \Delta$

7.2 The Horizontal Side-Pot Slicing Process (COMPUTEPOTS)

When multiple players are all-in for different amounts, the pot is partitioned into a Main Pot and one or more Side Pots by our horizontal commitment-slicing process (COMPUTEPOTS, Algorithm 17). All distinct positive commitment values $0 = L_0 < L_1 < L_2 < \cdots < L_k$ are collected across all players who put chips into the pot (including players who later folded). For each horizontal band $[L_{j-1}, L_j]$, every player p contributes the slice of their commitment that falls inside that band:

$$\gamma(p, L_{j-1}, L_j) = \max\Big(0,\ \min(p.\texttt{committed}, L_j) - \min(p.\texttt{committed}, L_{j-1})\Big). \tag{25}$$

The total chips in band j are $B_j = \sum_{p \in P} \gamma(p, L_{j-1}, L_j)$, the per-player contribution list is $C_j = \{(p, \gamma(p, L_{j-1}, L_j)) : \gamma > 0\}$, and the set of players **eligible to win** band j is:

$$E_j = \big\{p \in P : p.\texttt{dealtIn} \ \wedge\ \neg p.\texttt{folded} \ \wedge\ p.\texttt{committed} \geq L_j\big\}. \tag{26}$$

Algorithm 17: COMPUTEPOTS — slice multi-way commitments into horizontal bands and merge identical eligibility sets

Input: player array P

Output: ordered list of pots $\mathcal{P} = \big((amount_1, E_1, C_1), \ldots, (amount_r, E_r, C_r)\big)$

Extract strictly increasing positive commitment levels $L_1 < L_2 < \cdots < L_k$ from $\{p.\texttt{committed} > 0\}$

$\mathcal{P} \leftarrow [\,]$; $\quad prev \leftarrow 0$

for $j \leftarrow 1$ **to** k **do**

  $B_j \leftarrow \sum_{p \in P} \gamma(p, prev, L_j)$ via Eq. (25); record contributors $C_j \leftarrow \{(p, \gamma(p, prev, L_j)) : \gamma > 0\}$

  $E_j \leftarrow \{p.\texttt{id} : p.\texttt{dealtIn} \wedge \neg p.\texttt{folded} \wedge p.\texttt{committed} \geq L_j\}$

  if $B_j > 0$ **then**

    if $\mathcal{P} \neq [\,] \wedge \mathcal{P}_{\text{last}}.E = E_j$ **then** $\mathcal{P}_{\text{last}}.amount \mathrel{+}= B_j$; $\quad \mathcal{P}_{\text{last}}.C \leftarrow \mathcal{P}_{\text{last}}.C \cup C_j$ ▷ *merge adjacent bands*

    else append (B_j, E_j, C_j) to $\mathcal{P}$

  $prev \leftarrow L_j$

return $\mathcal{P}$

Proposition 7.1 (Exact Chip Conservation Across Horizontal Slices)**.** *Across all bands $j = 1, \ldots, k$, the sum of band amounts equals total committed chips: $\sum_{j=1}^{k} B_j = \sum_{p \in P} p.\texttt{committed}$. Furthermore, if all contributors at the highest band L_k have folded so that $E_k = \emptyset$, returning each contributor's slice $(p, c) \in C_k$ in* AWARDPOTS *conserves every chip exactly.*

Proof. For each individual player p with $p.\texttt{committed} = L_m$, the telescoping sum of Eq. (25) across $j = 1, \ldots, k$ is $\sum_{j=1}^{m}(L_j - L_{j-1}) + \sum_{j=m+1}^{k} 0 = L_m - 0 = p.\texttt{committed}$. Summing over all $p \in P$ proves $\sum_{j=1}^{k} B_j = \sum_{p \in P} p.\texttt{committed}$. □

Example 7.1 (Worked 4-Player Multi-Way Side-Pot Decomposition)**.** Suppose four players commit the following amounts during a hand:

- **Player A:** All-in for 100 (un-folded).
- **Player B:** Committed 200, then **folded** later in the hand.
- **Player C:** All-in for 350 (un-folded).
- **Player D:** Committed 350 (un-folded).

The distinct commitment levels are $L_1 = 100$, $L_2 = 200$, and $L_3 = 350$, with total pot $100 + 200 + 350 + 350 = 1,000$:

1. **Band 1 (**$0 \to 100$**):** All 4 players contribute 100 each $\implies$ $B_1 = 400$. Eligible winners: $E_1 = \{A, C, D\}$ (since B folded). This forms the **Main Pot of** 400 contested by $\{A, C, D\}$.
2. **Band 2 (**$100 \to 200$**):** Players B, C, D each contribute $200 - 100 = 100$ $\implies$ $B_2 = 300$. Eligible winners: $E_2 = \{C, D\}$.
3. **Band 3 (**$200 \to 350$**):** Players C, D each contribute $350 - 200 = 150$ $\implies$ $B_3 = 300$. Eligible winners: $E_3 = \{C, D\}$. Because $E_3 = E_2 = \{C, D\}$, Band 2 and Band 3 are merged into a single **Side Pot 1 of** $300 + 300 = 600$ contested by $\{C, D\}$.

7.3 The Showdown Evaluation, Reveal/Muck Order, and Odd-Chip Settlement Process (AWARDPOTS)

Once the pots $\mathcal{P}$ are constructed, our showdown settlement process (Algorithm 18) evaluates each un-folded player's 7-card hand once via BEST7$(p.H \cup B)$, determines the official showdown reveal/muck sequence (starting with the final street aggressor, or clockwise from left of the button when checked down, allowing beaten non-all-in AI hands to muck face-down), and awards each pot independently. If $m \geq 2$ eligible players tie for the highest score in a pot of size A, each tied winner receives $\lfloor A/m \rfloor$ chips, and the $r = A \bmod m$ indivisible **odd chips** (1 chip each) are awarded to the first r tied winners in **clockwise seat order starting immediately left of the dealer button** ($\texttt{btnSeat} + 1$), matching Poker TDA Rule 20.

Algorithm 18: AWARDPOTS — evaluate showdown hands, settle main and side pots, and distribute odd chips clockwise

Input: pots $\mathcal{P}$; un-folded players L; board B; dealer button seat `btnSeat`
Output: all pot chips transferred to winning player stacks (`pot` $\leftarrow 0$)

foreach $p \in L$ **do** $(p.\texttt{score}, p.\texttt{best5}) \leftarrow \text{BEST7}(p.H \cup B)$
foreach pot $(amount, E, C) \in \mathcal{P}$ **do**
  if $E = \emptyset$ **then foreach** $(p, c) \in C$ **do** $p.\texttt{stack} \mathrel{+}= c$; **continue** ▷ *refund orphaned slice*
  $v^\star \leftarrow \max_{p \in E} p.\texttt{score}$; $W \leftarrow \{p \in E : p.\texttt{score} = v^\star\}$
  Sort tied winners $W = (w_0, \ldots, w_{m-1})$ by clockwise distance $(w.\texttt{id} - \texttt{btnSeat} - 1) \bmod N_p$
  $q \leftarrow \lfloor amount / m \rfloor$; $r \leftarrow amount \bmod m$
  for $k \leftarrow 0$ **to** $m - 1$ **do** $w_k.\texttt{stack} \leftarrow w_k.\texttt{stack} + q + (1 \text{ if } k < r \text{ else } 0)$
`pot` $\leftarrow 0$

8 Process VI: Strategic Decision Policy, Nash Push/Fold, Tournament ICM, and Solver Processes

8.1 Player Archetypes and Difficulty Interpolation

To produce distinct, believable table personalities across our 4–8 player lineups while scaling cleanly across difficulty settings, each computer opponent is parameterized by a 3-dimensional **persona vector** (θ, α, β):

- $\theta \in [-0.14, 0.17]$ (**Tightness**): Minimum equity edge required above raw pot odds before calling, plus preflop hand-selection selectivity.
- $\alpha \in [0.20, 0.92]$ (**Aggression**): Propensity to raise rather than call or check with strong value hands.
- $\beta \in [0.03, 0.42]$ (**Bluff Frequency**): Propensity to bet or raise with weak holdings or semi-bluff draws.

Every newly dealt table is guaranteed to seat at least one Tight-Aggressive (TAG) anchor so short-handed lineups never degenerate into all maniacs or all calling stations. As difficulty increases from CASUAL ($\lambda = 0$) to SOLID ($\lambda = 0.35$) to SHARP ($\lambda = 0.70$), each seat's parameters are linearly pulled toward our disciplined baseline $(\theta_0, \alpha_0, \beta_0) = (0.05, 0.55, 0.12)$:

$$(\theta_{\text{eff}}, \alpha_{\text{eff}}, \beta_{\text{eff}}) = (1 - \lambda)(\theta, \alpha, \beta) + \lambda(\theta_0, \alpha_0, \beta_0). \tag{27}$$

8.2 The Preflop Heuristic Valuation Process (CHENSCORE)

In deep-stacked or multi-way preflop pots ($b_{\text{eff}} > 20$ big blinds or more than 2 live players), evaluating all 169 starting-hand classes quickly requires a closed-form heuristic. Bill Chen's starting-hand formula (CHENSCORE, Algorithm 19) scores any two hole cards (c_1, c_2) in $O(1)$ time by awarding base points to the higher card, doubling the score if paired (minimum 5), adding 2 points if suited, subtracting a gap penalty for disconnected ranks, adding 1 bonus point for compact low-gap connectors below a Queen, and taking the ceiling $\lceil s \rceil$.

Archetype	Code	θ	α	β	Behavioural Signature at the Table
Tight-Aggressive	TAG	$+0.05$	0.62	0.13	Selective preflop; enters pots primarily via raises and value bets strongly.
Loose-Aggressive	LAG	-0.07	0.80	0.30	Plays wide ranges, applies maximum bet pressure, and bluffs frequently.
Loose-Passive	CALL	-0.10	0.20	0.09	Calls wide preflop and postflop; rarely raises without a monster hand.
Ultra-Tight Nit	NIT	$+0.17$	0.30	0.03	Folds marginal hands early; enters pots almost exclusively with premium holdings.
Balanced Solid	SOLID	$+0.05$	0.55	0.12	Disciplined baseline toward which all archetypes converge at higher difficulty.

Table 4: Representative opponent persona parameters (θ, α, β) and their strategic tendencies.

Algorithm 19: CHENSCORE — closed-form preflop valuation of a two-card starting hand

Input: two hole cards c_1, c_2 with ranks $h = \max(\rho(c_1), \rho(c_2))$ and $\ell = \min(\rho(c_1), \rho(c_2))$
Output: Chen score $s \in [-1, 20]$

$s \leftarrow 10$ **if** $h = 12$ (A), 8 **if** $h = 11$ (K), 7 **if** $h = 10$ (Q), 6 **if** $h = 9$ (J), **else** $(h + 2)/2$
if $h = \ell$ **then return** $\lceil \max(5, 2s) \rceil$ ▷ *pocket pair (*A A $= 20$, K K $= 16, \ldots,$ 2 2 $= 5$*)*
if $\sigma(c_1) = \sigma(c_2)$ **then** $s \leftarrow s + 2$ ▷ *suited bonus*
$g \leftarrow h - \ell - 1$ ▷ *rank gap between the two cards*
$s \leftarrow s - \big(0 \text{ if } g = 0,\ 1 \text{ if } g = 1,\ 2 \text{ if } g = 2,\ 4 \text{ if } g = 3,\ \text{else } 5\big)$
if $g \leq 1 \wedge h < 10$ **then** $s \leftarrow s + 1$ ▷ *straight-connectivity bonus below Queen*
return $\lceil s \rceil$

8.3 The Short-Stack Nash Push/Fold Equilibrium Process (PUSHFOLD)

When effective stacks drop to $B_{\text{eff}} \leq 20$ big blinds in a heads-up or late-position blind-versus-blind confrontation ($n_{\text{live}} = 2$), postflop play shrinks and heuristic scoring becomes exploitable. Instead, our engine queries a **precomputed** 169×19 **Heads-Up Nash Equilibrium Push/Fold Table** (`PF_TABLE`, encoded as 6,422 hexadecimal nybbles in $\{0 \ldots 15\}$) solved offline via **Regret Matching** across the exact 169×169 preflop equity matrix $E[i, j]$ (incorporating card-removal weights $W[i, j]$).

In the push/fold subgame at stack depth $B \in \{2, 3, \ldots, 20\}$ big blinds (with small blind 0.5 bb and big blind 1.0 bb):

- If the Small Blind **folds**, the Small Blind's payoff is -0.5 bb.
- If the Small Blind **pushes all-in** with hand class i and the Big Blind **folds**, the Small Blind wins $+1.0$ bb (netting the Big Blind's posted 1 bb).
- If the Small Blind **pushes all-in** with hand class i and the Big Blind **calls** with hand class j, the total pot is $2B$ bb and the Small Blind's expected net payoff is $E[i, j] \cdot (2B) - B = B(2E[i, j] - 1)$.

Thus, given the Big Blind's calling probabilities $c_B(j) \in [0, 1]$ and the Small Blind's pushing probabilities $s_B(i) \in [0, 1]$, the exact expected values are:

$$\text{EV}_{\text{SB}}(\text{Push } i \mid c_B) = \frac{\sum_{j=0}^{168} W[i, j] \Big((1 - c_B(j)) \cdot 1.0 + c_B(j) \cdot B(2E[i, j] - 1) \Big)}{\sum_{j=0}^{168} W[i, j]}, \tag{28}$$

$$\text{EV}_{\text{BB}}(\text{Call } j \mid s_B) = \frac{\sum_{i=0}^{168} W[i, j]\, s_B(i) \Big((1 - E[i, j]) \cdot (2B) - (B - 1) \Big)}{\sum_{i=0}^{168} W[i, j]\, s_B(i)}. \tag{29}$$

At equilibrium, the Small Blind pushes whenever $\mathrm{EV}_{\mathrm{SB}}(\text{Push } i) > -0.5$, and the Big Blind calls whenever $\mathrm{EV}_{\mathrm{BB}}(\text{Call } j) > -1.0$. Table 5 lists the resulting combo-weighted push and call frequencies across stack depths, and Algorithm 20 specifies the $O(1)$ nybble lookup process.

Effective Stack (B)	SB Shove %	BB Call %	Representative Equilibrium Boundary Hands
20 bb	42.9%	22.0%	SB shoves 22+, A2s+, A7o+, K9s+, Q9s+, J9s+, T8s+, 98s; BB calls 44+, A8s+, ATo+, KJs+.
15 bb	51.2%	28.3%	SB widens to A2o+, K5s+, Q8s+, 87s, 76s; BB calls 33+, A5s+, A8o+, KTs+, KQo.
10 bb	61.3%	40.4%	SB shoves 61% of all hands (including K2s+, Q5s+, J7s+, 65s); BB calls 22+, A2s+, A4o+, K7s+, Q9s+.
5 bb	75.1%	68.2%	High dead-money leverage (1.5 bb in blinds vs. 5 bb stack); BB defends over two-thirds of hands.
2 bb	94.5%	100.0%	Getting 3 : 1 pot odds (1 bb to call into a 3 bb pot, needing only 25% equity), BB calls 100% of hands.

Table 5: Combo-weighted Heads-Up Nash Equilibrium shove and call percentages across stack depths.

Algorithm 20: PUSHFOLD — $O(1)$ hexadecimal nybble lookup in our 169×19 heads-up Nash push/fold table

Input: hole cards (c_1, c_2); effective stack in big blinds b_{eff}; role $which \in \{0 \text{ (shove)}, 1 \text{ (call)}\}$
Output: equilibrium probability $\pi^\star \in [0, 1]$ of shoving or calling an all-in

$r_1 \leftarrow \rho(c_1)$; $r_2 \leftarrow \rho(c_2)$; $h \leftarrow 12 - \max(r_1, r_2)$; $\ell \leftarrow 12 - \min(r_1, r_2)$
$(row, col) \leftarrow (h, \ell)$ **if** $r_1 = r_2 \vee \sigma(c_1) = \sigma(c_2)$ **else** (ℓ, h) ▷ *map to* 13×13 *grid*
$cls \leftarrow 13 \cdot row + col \in \{0, \ldots, 168\}$
$d \leftarrow \max\big(2,\ \min(20, \mathrm{round}(b_{\text{eff}}))\big) \in \{2, \ldots, 20\}$
return $\mathrm{parseInt}\big(\texttt{PF_TABLE}[(d-2) \cdot 338 + which \cdot 169 + cls],\ 16\big)/15$

8.4 The Malmuth–Harville Tournament ICM Process (COMPUTEICM)

When rising blinds are active in a tournament setting, chip expected value (cEV) is no longer proportional to monetary prize equity ($EV) because doubling a stack yields less than twice as much prize equity, whereas losing one's stack forfeits 100% of all future pay jumps. Under the **Malmuth–Harville Independent Chip Model (ICM)**, given survivor stacks $s = (s_1, \ldots, s_n)$ with total chips $S = \sum_{k=1}^{n} s_k$ and payout structure $w = (w_1, \ldots, w_m)$, player i finishes in 1st place with probability s_i / S and, conditional on player $j \neq i$ finishing 1st, competes recursively among $s \setminus \{s_j\}$ for prizes $(w_2, \ldots, w_m)$:

$$V_i(s; w_1, \ldots, w_m) = \frac{s_i}{S}\, w_1 + \sum_{j \neq i, s_j > 0} \frac{s_j}{S}\, V_i\big(s \setminus \{s_j\};\ w_2, \ldots, w_m\big). \tag{30}$$

When player i faces a substantial call of C chips into a pot P, let V_{win}, V_{fold}, and V_{lose} denote player i's ICM equity after winning the pot, folding, or losing the call. The tournament **Bubble Factor** $BF \geq 1.0$ and ICM break-even calling equity $q^\star_{\text{ICM}}$ are computed via Algorithm 21:

$$BF = \frac{V_{\text{fold}} - V_{\text{lose}}}{V_{\text{win}} - V_{\text{fold}}} \div \frac{C}{P} \geq 1.0, \qquad q^\star_{\text{ICM}} = \frac{BF \cdot C}{P + BF \cdot C}. \tag{31}$$

Algorithm 21: COMPUTEICM — recursive Malmuth–Harville tournament equity and survival Bubble Factor

Input: survivor stacks $s[0 \dots n-1]$; payout weights $w[0 \dots m-1]$; hero index h; aggressor index a; call C; pot P

Output: hero tournament equity $V_h(s)$, Bubble Factor $BF \geq 1.0$, and ICM calling threshold $q^\star_{\text{ICM}}$

Function ICM(s, w):
  Initialize $V[0 \dots n-1] \leftarrow 0$; recursively traverse finishing permutations of depth $d = 0 \dots m-1$ via Eq. (30)
  return V
Construct stack vectors $s_{\text{fold}}, s_{\text{win}}, s_{\text{lose}}$ for folding, winning P, or losing C to seat a
$V_{\text{fold}} \leftarrow \text{ICM}(s_{\text{fold}}, w)[h]$; $V_{\text{win}} \leftarrow \text{ICM}(s_{\text{win}}, w)[h]$; $V_{\text{lose}} \leftarrow \text{ICM}(s_{\text{lose}}, w)[h]$
$risk \leftarrow \max(0, V_{\text{fold}} - V_{\text{lose}})$; $gain \leftarrow \max(10^{-6}, V_{\text{win}} - V_{\text{fold}})$
$BF \leftarrow \text{clamp}_{[1.0,2.6]}\big((risk/gain)/(C/P)\big)$; $q^\star_{\text{ICM}} \leftarrow (BF \cdot C)/(P + BF \cdot C)$
return $(V_h(s), BF, q^\star_{\text{ICM}})$

8.5 The Preflop and Postflop Action Selection Processes

In multi-way or deep-stacked preflop situations, our preflop policy (PREFLOPDECISION) compares CHENSCORE$(p.H)$ against a position-calibrated threshold (calibrated to match empirical 6-max and 8-max raise-first-in frequencies from UTG through the Button), distinguishes opening the pot from 3-betting an existing raise, and defends the big blind wider against late-position steals.

On the flop, turn, and river, our postflop decision process (POSTFLOPDECISION, Algorithm 22) integrates five game-theoretic safeguards:

1. **Per-Opponent Multi-Way Range Floors ($c_{\text{floor}}(o)$) and Preflop Bunching:** For every active opponent o, an action-conditioned Chen floor $c_{\text{floor}}(o) \in [2.0, 10.5]$ is computed from the preflop raise count (2.5 unraised, 5.5 single-raised, 8.5 in 3-bet+ pots), persona tightness, preflop/street aggression, and Bayesian VPIP/PFR reads:

$$c_{\text{floor}}(o) = \text{clamp}_{[2.0,10.5]}\Big(c_{\text{base}}(\texttt{preflopRaises}) + 12\,\theta_{\text{eff}}(o) + \Delta_{\text{read}}(o) + 1.0 \cdot \mathbf{1}[\texttt{pfAggr} = o] + 0.8 \cdot \mathbf{1}[\texttt{aggr} = o \wedge \texttt{raises} \geq 2]\Big), \tag{32}$$

   and multi-way ranged equity is evaluated via EQUITYMULTIWAYRANGED (Algorithm 10).

2. **Board-Lock Split-Pot Protection:** If ISBOARDLOCKNUTCHOP (Algorithm 11) detects on the river that the 5-card board itself is the nuts ($\mathbb{P}(\text{loss}) = 0$), the engine never folds to an opponent's bet or shove.

3. **Square-Root Bet-Pressure Discount ($\hat{e}$):** Given $pressure = \texttt{toCall}/\max(\texttt{bb}, \texttt{pot})$, raw equity e is discounted by a concave square-root penalty and a multi-way penalty:

$$\hat{e} = e - \min\big(c_{\max},\ \gamma_{\text{press}}\sqrt{pressure}\big) - 0.025(m-1), \tag{33}$$

   where $(\gamma_{\text{press}}, c_{\max}) = (0.06, 0.10)$ when e is conditioned on narrowed opponent ranges via EQUITYMULTIWAYRANGED (preventing double-discounting of opponent range strength), or $(0.15, 0.20)$ against uniform ranges.

4. **Tournament ICM Survival Premium:** When rising blinds are enabled and a call risks $\geq 25\%$ of p's stack, the ICM survival premium $0.65(q^\star_{\text{ICM}} - q^\star_{\text{cEV}})$ from Algorithm 21 is added to the required calling threshold *need*.

5. **Chip-Step Bet Legalisation (CLAMPRAISE):** All proposed bet and raise sizes are rounded to natural casino chip steps (1, 5, or 25) and clamped into $[\tau_{\min}, \tau_{\max}]$.

Algorithm 22: POSTFLOPDECISION — multi-way range equity, board-lock chop guard, ICM bubble premium, and action selection

Input: player p; street state $(\texttt{toCall}, \texttt{pot}, b_{\text{behind}}, \texttt{canRaise}, \texttt{potOdds}, m, \texttt{reads})$
Output: legal postflop action $(a, \tau^\star)$

Compute $c_{\text{floor}}(o)$ via Eq. (32) for each active opponent $o \in \{0, \dots, m-1\}$
$(e, \text{SE}, p_{\text{win}}, p_{\text{tie}}, p_{\text{loss}}) \leftarrow \text{EQUITYMULTIWAYRANGED}(p.H, B, c_{\text{floor}}, n_{\text{fold}}, N_{\text{iters}})$
if $|B| = 5 \wedge \big(\text{ISBOARDLOCKNUTCHOP}(p.H, B) \vee (p_{\text{loss}} = 0 \wedge p_{\text{tie}} > 0.98)\big)$ **then**
  return (CALL if $\texttt{toCall} > 0$ else CHECK) ▷ *never fold a guaranteed chop*
if $\texttt{toCall} = 0$ **then** ▷ *checked to player p: choose between value bet, thin bet, bluff, or check*
  $\pi_{\text{bet}} \leftarrow 0.86$ if $e > 0.74$, $\min(0.92, 1.15\alpha_{\text{eff}} v_b)$ if $e > 0.62$, $\min(0.90, 0.42\alpha_{\text{eff}} v_b)$ if $e > 0.50$, else $\min(0.85, \beta_{\text{eff}} m_b)$
  if $\texttt{canRaise} \wedge U < \pi_{\text{bet}}$ **then return** $(\text{RAISE}, \text{CLAMPRAISE}(p, \text{SIZEBET}(e)))$ **else return** CHECK
pressure $\leftarrow \texttt{toCall}/\max(\texttt{bb}, \texttt{pot})$; $\hat{e} \leftarrow e - \min\big(c_{\max}, \gamma_{\text{press}}\sqrt{\textit{pressure}}\big) - 0.025(m-1)$
need $\leftarrow \texttt{potOdds} + \theta_{\text{eff}} + 0.014\, b_{\text{behind}} + 0.07 \cdot \mathbf{1}[\texttt{raises} \geq 3] + 0.05 \cdot \mathbf{1}[\texttt{toCall} \geq p.\texttt{stack}] + \Delta_{\text{ICM}} - \Delta_{\text{callDown}} - \Delta_{\text{learned}}$
if $e > 0.72 \wedge \texttt{canRaise} \wedge U < 0.92\alpha_{\text{eff}}$ **then return** $(\text{RAISE}, \text{CLAMPRAISE}(p, \text{SIZEBET}(e)))$ ▷ *value raise*
if $\hat{e} \geq$ *need* **then return** CALL
if $\texttt{canRaise} \wedge \texttt{raises} < 2 \wedge e > 0.28 \wedge U < 0.4\beta_{\text{eff}} m_b$ **then return** $(\text{RAISE}, \text{CLAMPRAISE}(p, \text{SIZEBET}(0.6)))$ ▷ *semi-bluff*
if $\texttt{toCall} \leq \texttt{bb} \wedge \hat{e} \geq \textit{need} - 0.08$ **then return** CALL
return FOLD

8.6 Processes in Associated Full-Game GTO Solvers (CFR, Abstraction, and Subgame Solving)

In large-scale offline and cloud poker solvers (such as `Slumbot`, `Libratus`, `Pluribus`, and commercial GTO solver suites), approximate Nash equilibria are computed across the entire multi-street game tree of No-Limit Hold'em (which contains over 10^{160} decision nodes) via a three-stage **Counterfactual Regret Minimization (CFR) Process**:

1. **Stage 1 — Information and Action Abstraction:** To shrink the 10^{160}-node tree to a tractable 10^7–10^9 information sets:
   - *Card Bucketing (K-Means over Earth Mover's Distance):* Hands on the flop, turn, and river are clustered into $K \in [200, 5000]$ buckets based on their equity distribution across future board runouts.
   - *Action Discretization:* Continuous No-Limit bet sizes are restricted to a sparse grid of pot fractions (e.g., $\{0.33\times, 0.50\times, 0.75\times, 1.0\times, 1.5\times, \text{All-In}\}$), with off-tree human bet sizes mapped to nearby grid points via randomized pseudo-harmonic translation.
2. **Stage 2 — Counterfactual Regret Minimization (CFR) and Regret Matching:** An **information set** I groups all game-tree histories that are indistinguishable to the acting player i (same private cards and same public betting/board sequence). Let $\sigma^t(I, a)$ denote the probability

assigned to action $a \in A(I)$ on iteration t, and let $v_i(\sigma^t, I)$ denote player i's expected counterfactual utility at I. The **instantaneous counterfactual regret** of not taking action a at I on iteration t is:

$$r^t(I,a) = v_i\big(\sigma^t_{I\to a}, I\big) - v_i\big(\sigma^t, I\big), \qquad R^T(I,a) = \sum_{t=1}^{T} r^t(I,a). \tag{34}$$

At iteration $T+1$, **Regret Matching** updates the next strategy $\sigma^{T+1}(I,a)$ proportionally to the positive cumulative regrets $R^T_+(I,a) = \max(0, R^T(I,a))$:

$$\sigma^{T+1}(I,a) = \begin{cases} \dfrac{R^T_+(I,a)}{\sum_{a'\in A(I)} R^T_+(I,a')} & \text{if } \sum_{a'\in A(I)} R^T_+(I,a') > 0, \\ \dfrac{1}{|A(I)|} & \text{otherwise.} \end{cases} \tag{35}$$

By Zinkevich et al.'s theorem, as $T \to \infty$, overall average regret decays as $O(1/\sqrt{T})$, and the **cumulative average strategy profile** $\bar{\sigma}^T(I,a)$:

$$\bar{\sigma}^T(I,a) = \frac{\sum_{t=1}^{T} \pi_i^{\sigma^t}(I)\, \sigma^t(I,a)}{\sum_{t=1}^{T} \pi_i^{\sigma^t}(I)} \tag{36}$$

converges to an exact Nash equilibrium in any two-player zero-sum game.

3. **Stage 3 — Real-Time Depth-Limited Subgame Solving:** When an off-tree bet is made postflop, the subgame rooted at the current public state is reconstructed using both players' Bayesian range distributions and solved in real time over the unabstracted turn/river subgame.

Engine / Solver	Primary Process	Memory / Speed	Best-Suited Application Domain
Our Reference Engine	Nash Push/Fold + Multi-Way Range MC + ICM + Bayesian Reads	< 1 MB, < 25 ms	Instant zero-dependency 4–8 player play with provably fair enclave and adaptive exploitation.
Rules Engines (`PokerKit`)	Pure multi-variant state validation	< 5 MB, < 1 ms	Backend hand-history parsing, multi-game rule enforcement, RL environments.
Blueprint CFR (`Slumbot`)	Offline MCCFR over card/bet abstractions	10–100 GB RAM	Fast cloud lookup of full-tree approximate GTO strategies.
Subgame Solvers (`Pluribus`)	Blueprint + real-time depth-limited CFR	Multi-core CPU cluster	Superhuman unexploitable play in deep-stacked cash and 6-max games.
Deep RL (`AlphaHoldem`)	Trinal-Clip PPO end-to-end neural policy	GPU inference (4 ms)	High-speed neural evaluation without explicit abstraction tables.

Table 6: Architectural comparison across poker rules engines, real-time heuristic/Monte Carlo engines, and full-game GTO solvers.

9 Process VII: Bayesian Opponent Profiling and Hindsight-Free Self-Review

9.1 The Beta-Binomial Updating Process and Two-Sigma Gate

A purely static equilibrium strategy never takes full advantage of a human opponent who folds 80% of the time to postflop bets or calls down with any pair. Conversely, when raw percentages are computed after just 3 hands, a naive adaptive bot will wildly over-adjust to small-sample noise.

In our Bayesian opponent-profiling process (BAYESIANREAD, Algorithm 23), each binary human tendency (`vpip`, `pfr`, `foldToRaise`, `foldToBet`, and `aggr`) is modelled as a **Beta-Binomial conjugate prior** $\mathrm{Beta}(a_0, b_0)$ with prior mean $\mu_0 = \frac{a_0}{a_0+b_0}$ (specifically, `vpip` $\sim \mathrm{Beta}(5,7)$ with $\mu_0 \approx 41.7\%$, `pfr` $\sim \mathrm{Beta}(3,11)$ with $\mu_0 \approx 21.4\%$, `foldToRaise` $\sim \mathrm{Beta}(6,6)$ with $\mu_0 = 50.0\%$, `foldToBet` $\sim \mathrm{Beta}(6,6)$ with $\mu_0 = 50.0\%$, and `aggr` $\sim \mathrm{Beta}(4,10)$ with $\mu_0 \approx 28.6\%$). After k occurrences are observed out of n_{obs} opportunities, the exact posterior distribution is $\mathrm{Beta}(a, b)$ with $a = a_0 + k$ and $b = b_0 + (n_{\text{obs}} - k)$, whose posterior mean μ and posterior standard deviation σ are:

$$\mu = \frac{a}{a+b}, \qquad \sigma = \sqrt{\frac{a\,b}{(a+b)^2\,(a+b+1)}}. \tag{37}$$

To prevent small-sample overfitting, a non-zero exploitation signal $z \in [-1,1]$ is emitted **if and only if the posterior mean deviates from the prior mean by more than two posterior standard deviations** ($|\mu - \mu_0| > 2\sigma$):

$$z = \begin{cases} \mathrm{clamp}_{[-1,1]}\left(\dfrac{\mu - \mu_0}{4\sigma}\right) & \text{if } |\mu - \mu_0| > 2\sigma, \\ 0 & \text{otherwise.} \end{cases} \tag{38}$$

Algorithm 23: BAYESIANREAD — Beta-Binomial posterior update, 2σ significance gate, and context-gated exploitative modifiers

Input: prior (a_0, b_0); current posterior (a, b); interaction context (whether target opponent faces the action)

Output: significance-gated signal $z \in [-1,1]$ and exploitative multipliers $(m_b, v_b, \Delta_{\text{callDown}}, \mathit{stealDrop})$

$\mu_0 \leftarrow a_0/(a_0+b_0)$; $\quad n \leftarrow a+b$; $\quad \mu \leftarrow a/n$

$\sigma \leftarrow \sqrt{a\,b/\left(n^2(n+1)\right)}$

$z \leftarrow \mathrm{clamp}_{[-1,1]}\left((\mu-\mu_0)/(4\sigma)\right)$ **if** $|\mu-\mu_0| > 2\sigma$ **else** 0 ▷ 2σ *credible-interval gate*

$m_b \leftarrow \max(0.05, 1 + 1.6\,z_{\text{foldToBet}})$ and $v_b \leftarrow 1 - 0.30\,z_{\text{foldToBet}}$ **if** target opponent can face a bet **else** $(1,1)$

$\Delta_{\text{callDown}} \leftarrow 0.06\max(0, z_{\text{aggr}})$ **if** target opponent is current street aggressor **else** 0

$\mathit{stealDrop} \leftarrow 1.6\max(0, z_{\text{foldToRaise}})$ **if** target opponent has not acted preflop **else** 0

return $(z, m_b, v_b, \Delta_{\text{callDown}}, \mathit{stealDrop})$

9.2 The Hindsight-Free Post-Showdown Self-Review Process (REVIEWHAND)

In addition to profiling opponents, how can our engine calibrate its own calling thresholds across a session without cheating? Two classic design mistakes corrupt naive self-learning poker bots:

1. **Outcome Bias (Result-Oriented Learning):** When a call is judged by whether the final river card happened to win or lose the pot, the bot is taught to fold 80% favourites after a single bad beat.
2. **Information Leakage:** When the hole cards of opponents who folded or mucked are inspected after the hand, illicit information is leaked to the AI that a real player at the table is never permitted to see.

Both errors are avoided by our post-showdown self-review process (REVIEWHAND, Algorithm 24): every decision facing a bet (`toCall` > 0) is recorded into a trace buffer during the hand, and at

the end of the hand the **expected value at the moment of the decision** is evaluated using *only* the hole cards of players who actually tabled their cards at showdown ($q_{\text{true}} = \text{EQUITYVSKNOWN}$), while any folded or mucked opponents are re-sampled as unknown random hands from the unseen stub. The resulting counterfactual regret (in big blinds) updates a coarse bucket key $(\texttt{stage}, \text{equityTier}, \text{oddsTier})$ capped at ± 0.07 threshold shift after at least 15 samples.

Algorithm 24: REVIEWHAND — hindsight-free post-showdown expected-value regret update without peeking at mucked cards

Input: set of players *shown* whose cards were tabled at showdown; decision trace buffer $\mathcal{T}$
Output: updated regret buckets $\mathcal{L}[k] = (r_{\text{fold}}, r_{\text{call}}, n)$

$S_{\text{id}} \leftarrow \{q.\texttt{id} : q \in \textit{shown}\}$
foreach decision record $d = (\textit{hole}, \textit{board}, \textit{oppIds}, \texttt{pot}, \texttt{toCall}, \textit{act}, k) \in \mathcal{T}$ **do**
  $H_{\text{known}} \leftarrow \{p_i.H : i \in \textit{oppIds} \cap S_{\text{id}}\}$; $u_{\text{mucked}} \leftarrow |\textit{oppIds} \setminus S_{\text{id}}|$ ▷ *mucked cards stay hidden*
  $q_{\text{true}} \leftarrow \text{EQUITYVSKNOWN}(\textit{hole}, H_{\text{known}}, \textit{board}, u_{\text{mucked}})$
  $\text{EV}_{\text{fold}} \leftarrow 0$; $\text{EV}_{\text{call}} \leftarrow q_{\text{true}} \cdot \texttt{pot} - (1 - q_{\text{true}}) \cdot \texttt{toCall}$
  $\textit{base} \leftarrow (\text{EV}_{\text{fold}} \text{ if } \textit{act} = \text{FOLD else } \text{EV}_{\text{call}})$
  $\mathcal{L}[k].r_{\text{fold}} \mathrel{+}= (\text{EV}_{\text{fold}} - \textit{base})/\texttt{bb}$; $\mathcal{L}[k].r_{\text{call}} \mathrel{+}= (\text{EV}_{\text{call}} - \textit{base})/\texttt{bb}$; $\mathcal{L}[k].n \mathrel{+}= 1$
Clear trace buffer $\mathcal{T} \leftarrow [\,]$

10 System Invariants, Process Complexity, and Worked Walkthrough

10.1 The Seven Formal System Invariants

Across every deal, action, side-pot split, and showdown in our engine and companion Python verification suite, seven mathematical invariants are enforced and verified at all times:

Our Seven Formal Engine Invariants.

1. **Chip Conservation (Invariant I1):** Let $M_0 = \sum_p p.\texttt{stack}_0$ be total table chips at the start of a hand. At every instant during and after the hand, $\sum_p p.\texttt{stack} + \texttt{pot} = M_0$, and during active play $\texttt{pot} = \sum_p p.\texttt{committed}$.
2. **Non-Negative Stack and Commitment Bounds (Invariant I2):** For every player p, $p.\texttt{stack} \geq 0$, $p.\texttt{bet} \geq 0$, and $p.\texttt{committed} \geq p.\texttt{bet}$. Moreover, $p.\texttt{allIn} = \textbf{true} \iff (\neg p.\texttt{folded} \wedge p.\texttt{stack} = 0 \wedge p.\texttt{committed} > 0)$.
3. **Deck Card Bijection and Enclave Seal (Invariant I3):** At every street, the multiset union of remaining stub cards in DEALERVAULT, burned cards, board cards B, and all dealt player hole cards $p.H$ has cardinality 52 and equals $\{0, 1, \ldots, 51\}$ without duplicates, matching the pre-hand SHA-256 commitment seal.
4. **Evaluator Monotonicity and Equivalence (Invariant I4):** For every 7-card subset $A \subset \{0, \ldots, 51\}$, $\textsc{Rank7Fast}(A) = \textsc{Best7}(A) = \max_{C \subset A, |C|=5} \textsc{Eval5}(C)$, and $\textsc{Decode}(\textsc{Encode}(\kappa, k)) = (\kappa, k)$.
5. **Turn-Order Termination (Invariant I5):** Every betting round terminates in a finite number of actions ($\leq N_p(1 + N_{\text{raises}})$), because every action either increments the count of players with $\texttt{hasActed} = \textbf{true}$, folds a player, puts a player all-in, or strictly increases $\texttt{currentBet}$ toward the finite table chip supply M_0.
6. **Side-Pot Eligibility Soundness (Invariant I6):** No player p can ever be eligible for any horizontal commitment band $[L_{j-1}, L_j]$ where $L_j > p.\texttt{committed}$, nor can any folded player belong to any eligibility set E_j.
7. **Statistical Gating and Hindsight Isolation (Invariant I7):** All Bayesian opponent adjustments equal 0 whenever $|\mu - \mu_0| \leq 2\sigma$, and post-hand review never accesses the hole cards of folded or mucked players.

10.2 Summary of Process Complexity and Runtime Benchmarks

In Table 7, we summarise the asymptotic time complexity, space footprint, and measured single-thread execution latency of each core algorithm in our engine.

10.3 Complete End-to-End Numerical Walkthrough of a 4-Player Hand Across All Seven Processes

To see how all seven processes work together seamlessly during a single hand, we trace a 4-player table (the minimum starting table size in our engine: Seat 0: TAG, Seat 1: HUMAN, Seat 2: NIT, Seat 3: LAG) with blinds $\texttt{sb/bb} = 10/20$ and stacks $(1000, 1000, 1000, 240)$:

1. **Process I & IV (TDA Rule 19 Blind Advancement, Enclave Sealing & Dealing):** Suppose Seat 0 posted the big blind on the prior hand ($\texttt{prevBbSeat} = 0$). By Algorithm 13 (TDA Rule 19), $\texttt{sbSeat}$ is anchored directly to Seat 0 (10 chips), $\texttt{bbSeat}$ advances clockwise to Seat 1 (20 chips), $\texttt{btnSeat}$ is placed at Seat 3, and Seat 2 is badged as $\texttt{UTG}$. Inside DEALERVAULT (Algorithm 2), the 52-card deck is permuted, the salted SHA-256 commitment seal is published, and two cards are dealt clockwise one at a time starting at Seat 0: Seat 0 (SB) receives J♠ J♡ ($\{36, 37\}$), Seat 1 (BB) receives Q♠ T♠ ($\{40, 32\}$), Seat 2 (UTG) receives 7♢ 2♣ ($\{22, 3\}$), and Seat 3 (BTN) receives A♡ K♢ ($\{49, 46\}$). Total pot is $\texttt{pot} = 30$, $\texttt{currentBet} = 20$, $\texttt{lastRaise} = 20$, and $\texttt{actingIdx} = 2$ (UTG).

Process / Algorithm	Time Complexity	Space	Measured Latency	Verification & Operational Role
CRYPTOGRAPHICSHUFFLE + DEALERVAULT	$O(52)$	$O(52)$	0.90 ms / hand	Rejection-sampled Fisher–Yates over 52! decks + SHA-256 seal.
ENCODE/DECODE	$O(1)$ (5 steps)	$O(1)$	< 15 ns	Bijective base-13 score packing $(\kappa \cdot 13^5 + \dots)$.
BEST7 (EVAL5 $\times$ 21)	$O(\binom{7}{5}) = 21$	$O(1)$	41.7 μs / hand	Showdown oracle; identifies exact 5 winning cards.
RANK7FAST	$O(1)$ (1 pass)	48 KB	**564** ns / hand	1.77M evals/s; 0 mismatches vs. BEST7 on 200k deals.
EQUITY (N trials)	$O(N \cdot m)$	$O(52)$	1.9 ms ($N = 600$)	Partial Fisher–Yates rollout + Bessel $(N-1)$ pot-share SE.
EQUITYMULTIWAYRANGED	$O(N_{\text{att}} \cdot m)$	10.8 KB	3.8 ms ($N = 900$)	Multi-way range rejection sampling + preflop fold bunching.
ISBOARDLOCKNUTCHOP	$\binom{45}{2} = 990$	$O(45)$	0.55 ms	River chop guard; verifies $\mathbb{P}(\text{loss}) = 0$ across all 990 pairs.
NEXTCARDSCAN (Turn HU)	$46 \times \binom{45}{2}$	$O(46)$	24.4 ms (exact)	Enumerates all 45,540 deals with SE $= 0$.
COMPUTEPOTS + AWARDPOTS	$O(N_p^2)$	$O(N_p)$	< 0.05 ms	Horizontal commitment slicing, contributor refund & odd chips.
PUSHFOLD + COMPUTEICM	$O(1)$ / $O(N_p!)$	6.4 KB	< 0.15 ms	169×19 HU Nash lookup ($\leq$ 20 bb) + recursive Malmuth–Harville ICM.

Table 7: Time complexity, memory footprint, and benchmarked execution latencies across our engine processes.

2. **Process VI (Preflop Valuation & Action):** Seat 2 acts first under the gun with 7♢ 2♣; CHENSCORE$(7\diamondsuit, 2\clubsuit) = \lceil 3.5 - 5 \rceil = -1 < t_{\text{open}}$, so Seat 2 folds. Seat 3 acts on the button with A♡ K♢ (CHENSCORE $= 10 \geq t_{\text{open}}$) and raises to 55. Seat 0 (J♠ J♡, CHENSCORE $= 12$) calls 45 more (to 55), and Seat 1 (Q♠ T♠, CHENSCORE $= 8$) calls 35 more (to 55). All three remaining players now have `hasActed` = **true** and `bet` $= 55$, so FINDNEXTACTOR returns -1. The street ends with `pot` $= 165$.

3. **Process II, III & VI (Flop Evaluation, Multi-Way Range Equity & Short All-In):** One burn card is discarded inside DEALERVAULT and the flop comes $B = (\text{Q}\heartsuit, 8\spadesuit, 4\clubsuit)$. Seat 0 checks (`actedAtBet` $= 0$), and Seat 1 (top pair, Q♠ T♠) bets 100 (`currentBet` $= 100$, `lastRaise` $= 100$, p_1.`actedAtBet` $= 100$). Seat 3 (A♡ K♢, remaining stack $240 - 55 = 185$) moves all-in for 185. Because the raise increment is $\Delta = 185 - 100 = 85 <$ `lastRaise` $= 100$, this is a **short all-in**: `currentBet` becomes 185, while `lastRaise` remains 100.

   - Action returns to Seat 0 (J♠ J♡). Because Seat 0 checked at p_0.`actedAtBet` $= 0$, CANREOPEN(Seat 0) checks $185 - 0 = 185 \geq 100 \implies$ **true**. Seat 0 runs

POSTFLOPDECISION (using Seat 2's folded bunching and Seat 1/Seat 3's range floors), sees $\hat{e} < need$ in a 3-way pot facing a bet and a shove, and **folds** (leaving 55 dead chips in the pot).

- Action returns to Seat 1 ($\texttt{bet} = 100 < 185$). Checking CANREOPEN(Seat 1) yields $185 - 100 = 85 < \texttt{lastRaise} = 100 \implies$ **false**: Seat 1 is only permitted to call the additional 85 or fold. Seat 1 calls 85. The pot is now $\texttt{pot} = 55 + 240 + 240 = 535$.

4. **Process III (Turn Exact Enumeration):** Because Seat 3 is all-in ($n_{\text{canAct}} = 1$ and $p_1.\texttt{bet} = 185 = \texttt{currentBet}$), the turn 2♢ and river 7♣ are dealt out automatically. On the turn $B_4 =$ (Q♡, 8♠, 4♣, 2♢), NEXTCARDSCAN (Algorithm 12) enumerates all $46 \times 990 = 45,540$ river deals with SE $= 0$; against Seat 3's actual A♡ K♢, Seat 3 needs one of the 3 remaining Aces or 3 remaining Kings (6 outs) out of 44 unseen cards, giving exact river equity $6/44 = 13.64\%$, while Seat 1 (Q♠ T♠) holds 86.36% equity.

5. **Process V & VII (Side-Pot Slicing, Showdown Settlement & Hindsight-Free Review):** After the river 7♣ falls, commitments are $p_0.\texttt{committed} = 55$ (folded), $p_1.\texttt{committed} = 240$ (live), $p_2.\texttt{committed} = 0$ (folded), and $p_3.\texttt{committed} = 240$ (live).

   - REFUNDUNCALLED sees $c_1 = c_2 = 240$, so $\Delta_{\text{uncalled}} = 0$.
   - COMPUTEPOTS slices Band 1 ($0 \to 55$, amount $3 \times 55 = 165$, $E_1 = \{1, 3\}$) and Band 2 ($55 \to 240$, amount $2 \times 185 = 370$, $E_2 = \{1, 3\}$), and merges them because $E_1 = E_2 = \{1, 3\}$ into a single **Main Pot of** 535.
   - Because Seat 3 is all-in, both Seat 3 and Seat 1 table their hands at showdown. AWARDPOTS scores Seat 1's Pair of Queens ($\kappa = 1, k = (10, 8, 6, 5)$, score $V = 675,558$) above Seat 3's Ace-High ($\kappa = 0, k = (12, 11, 10, 6, 5)$, score $V = 368,672$), awarding all 535 chips to Seat 1.
   - Finally, REVIEWHAND updates post-decision regret buckets using Seat 1 and Seat 3's revealed showdown hands while keeping Seat 0's folded flop hand (J♠ J♡) hidden, and DEALERVAULT.VERIFYSEAL() confirms the 52-card SHA-256 commitment seal. Total table chips remain $945 + 1295 + 1000 + 0 = 3240 = M_0$ (Invariant I1).